

NAT 1

Documento Técnico Oficial

Produto, Regras de Negócio, MVP 1, Requisitos, RNF e Modelagem Inicial

v1.0 — Consolidado para início do desenvolvimento

Data: 04/06/2026

Ferramenta de organização, memória viva e auxílio à mestragem para RPG de mesa

Preparado para Gabriel Bomfim Bispo

Controle do Documento

Documento	Nat 1 — Documento Técnico Oficial
Versão	v1.0 — Consolidado para início do desenvolvimento
Objetivo	Consolidar as decisões aprovadas do produto Nat 1 para orientar a implementação do MVP 1.
Base utilizada	Texto consolidado enviado pelo usuário com RN, RF, RNF e modelagem inicial.
Status	Aprovado como base de codificação após revisão de engenharia.
Observação	Este documento preserva as decisões aprovadas e aplica ajustes de consistência terminológica e técnica sem remover conteúdo essencial.

Revisão de Engenharia Aplicada

A consolidação abaixo foi revisada como documentação técnica inicial do Nat 1. O objetivo foi manter o material aprovado, ajustar inconsistências de nomenclatura e deixar a base pronta para implementação gradual. Nenhuma regra de negócio aprovada foi removida.

- Mantida a separação entre interface "Campanhas & Crônicas" e conceito técnico "Projeto de Jogo".
- Padronizado o entendimento de "Facções" como módulo mais amplo "Organizações / Facções" na modelagem, preservando o termo Facções onde ele já aparece nas RN e RF.
- Mantida a separação entre Tipo principal, Natureza/Classificação e Espécie/Raça/Povo/Linhagem em Personagens & Criaturas.
- Mantida a decisão de não usar uma tabela genérica "entities" no MVP 1; o projeto começa com tabelas claras por módulo principal.
- Preservada a relação polimórfica controlada para conectar conteúdos do mundo vivo.
- Mantidos tags e coleções como opcionais do MVP 1.2, não como bloqueio da primeira codificação.
- Mantidos eventos, calendários, timelines, mapas avançados, diários/recaps, IA/RAG e jogadores/permissões como fases futuras.
- Reforçada a necessidade de validação forte no backend, isolamento por usuário, migrations e estrutura preparada para temas e permissões futuras.

Resumo Executivo para Codificação

Stack aprovada: Frontend: React + TypeScript + Vite, Tailwind CSS, React Router, TanStack Query, React Hook Form e Zod. Backend: FastAPI + Python, PostgreSQL, SQLAlchemy, Alembic e Pydantic.

MVP 1: Versão organizacional para mestres/narradores, sem IA pesada, RAG, jogadores, chat, mapas interativos avançados ou exportações complexas.

Núcleo do MVP: Autenticação, Home do Mestre, Campanhas & Crônicas, Dashboard do Projeto, módulos internos essenciais, relações básicas, documentos simples e notas.

Modelagem inicial: users, game_projects, worlds, system_templates, project_module_settings, sessions, scenes, character_creatures, locations, organizations_factions, documents, notes e relationships.

Próximo passo técnico: Definir arquitetura de pastas, contratos de API e primeira sequência de implementação.

Sumário Geral

1. 1. Regras de Negócio Consolidadas - RN-001 a RN-092
2. 2. Requisitos Funcionais do MVP 1 - Parte 1
3. 3. Requisitos Funcionais do MVP 1 - Parte 2
4. 4. Requisitos Não Funcionais do MVP 1
5. 5. Modelagem Inicial de Dados do MVP 1
6. 6. Revisão baseada em modularização de ferramentas de campanha/worldbuilding

Regras de Negócio Consolidadas

Versão consolidada — RN-001 a RN-092

Parte 1 — Fundamentos, Usuário, Campanhas & Crônicas, Mundos e Sistemas

RN-001 — O Nat 1 é uma ferramenta para mestres/narradores

O Nat 1 é uma plataforma de organização, memória viva e auxílio à mestragem para RPG de mesa.

Na primeira versão, o produto será focado no mestre/narrador, permitindo que ele organize campanhas, crônicas, mundos, sessões, cenas, personagens, criaturas, locais, facções, documentos, notas e relações.

A experiência dos jogadores será planejada para fases futuras, mas a arquitetura deve nascer preparada para papéis, permissões, visibilidade limitada e expansão gradual.

RN-002 — O usuário principal da Fase 1 é o mestre

Na Fase 1, o usuário principal será o mestre/narrador responsável pela organização da mesa.

O sistema deve permitir que esse usuário:

- crie e organize Campanhas & Crônicas;
- cadastre mundos/cenários;
- escolha sistemas/templates;
- registre sessões e cenas;
- organize personagens, criaturas, locais e facções;
- crie notas e documentos;
- estabeleça relações entre conteúdos;
- configure módulos ativos da campanha;
- preserve informações públicas e privadas.

Jogadores não terão acesso funcional no MVP 1, mas a estrutura deve prever esse tipo de usuário no futuro.

RN-003 — O Nat 1 usa “Campanhas & Crônicas” como nome amigável na interface

Na interface principal, o produto deve usar o termo Campanhas & Crônicas para representar os projetos de mesa do usuário.

Esse termo foi escolhido por ser mais familiar para mestres de RPG e abranger tanto jogos de fantasia medieval quanto jogos narrativos como Vampiro: A Máscara.

Exemplos:

- campanha de D&D;
- crônica de Vampiro;
- caso de Call of Cthulhu;
- missão de Ordem Paranormal;
- one-shot autoral;
- mini-campanha;
- aventura curta;
- sandbox.

RN-004 — Internamente, Campanhas & Crônicas são tratadas como Projetos de Jogo

Apesar do nome amigável na interface, o conceito técnico interno será Projeto de Jogo.

Um Projeto de Jogo é qualquer estrutura jogável organizada pelo mestre, podendo representar:

- campanha;
- crônica;
- caso;
- investigação;
- aventura;
- one-shot;
- mini-campanha;

- sandbox.

Essa separação permite que a interface use nomes adequados ao sistema, sem comprometer a consistência técnica do produto.

RN-005 — Um usuário pode ter múltiplos Projetos de Jogo

O usuário poderá criar, editar, arquivar e acessar múltiplos Projetos de Jogo.

Cada Projeto de Jogo deve funcionar como um espaço próprio, contendo suas próprias configurações, tema visual, sistema, módulos ativos, documentos, sessões, cenas, entidades e relações.

Exemplos:

- Blood and Business — Crônica de Vampiro: A Máscara;
- Aelthos — Campanha de D&D 5e;
- O Farol Sem Lua — Caso de Call of Cthulhu;
- Caso Aracaju — Missão de Ordem Paranormal;
- A Mina Sem Nome — One-shot autoral.

RN-006 — O Projeto de Jogo deve ter um formato definido

Ao criar um novo Projeto de Jogo, o usuário deve poder escolher seu formato.

Formatos iniciais previstos:

- Campanha;
- Crônica;
- Caso / Investigação;
- Aventura;
- One-shot;
- Mini-campanha;
- Sandbox;
- Outro / Personalizado.

O formato ajuda a adaptar a linguagem da interface e a sugerir módulos, mas não deve limitar o funcionamento do sistema.

Exemplo: uma Crônica de Vampiro e uma Campanha de D&D são estruturalmente equivalentes no Nat 1, mas podem usar nomenclaturas e templates diferentes.

RN-007 — A nomenclatura da interface pode variar conforme o sistema ou preferência do mestre

O Nat 1 deve evitar rigidez terminológica quando diferentes sistemas usam nomes diferentes para conceitos equivalentes.

Exemplos:

- “Campanha” em D&D pode equivaler a “Crônica” em Vampiro;
- “Mestre” pode equivaler a “Narrador”, “Guardião”, “DM” ou “GM”;
- “Personagem” pode equivaler a “Investigador”, “Cainita”, “Agente” ou “Aventureiro”;
- “Grupo” pode equivaler a “Coterie”, “Party”, “Equipe” ou “Mesa”.

A interface deve permitir adaptação por sistema/template sem alterar a estrutura interna do produto.

RN-008 — A Home do Mestre é o ponto de entrada global

Após login, o usuário deve acessar a Home do Mestre, e não diretamente uma campanha específica.

A Home do Mestre deve apresentar uma visão global, limpa e organizada, com acesso a:

- continuar última campanha/crônica;
- campanhas recentes;
- criar novo Projeto de Jogo;
- mundos/cenários;
- sistemas e templates;
- biblioteca global;
- notas rápidas;
- configurações.

A Home do Mestre não deve exibir como navegação principal módulos internos como personagens, criaturas, sessões, mapas, itens ou facções, pois esses conteúdos pertencem a um Projeto de Jogo, Mundo/Cenário ou Sistema específico.

RN-009 — A tela global deve ter navegação simples

A navegação global deve conter poucos itens principais para evitar poluição visual.

Menu global recomendado:

- Home;
- Campanhas & Crônicas;
- Mundos / Cenários;
- Sistemas & Templates;
- Biblioteca Global;
- Notas rápidas;
- Configurações.

Essa navegação representa o workspace geral do mestre.

RN-010 — O usuário precisa entrar em um Projeto de Jogo para acessar seus módulos internos

Os módulos ricos da mesa só devem aparecer depois que o usuário entrar em uma Campanha/Crônica específica.

Dentro do Projeto de Jogo, o usuário poderá acessar módulos como:

- Resumo;
- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais;
- Facções;
- Itens & Equipamentos;
- Magias & Poderes;
- Regras / Compêndio;
- Mapas;
- Documentos;
- Handouts;
- Pistas & Segredos;
- Relações;
- Notas;
- Recursos / Soundtracks;
- IA da Campanha;
- Configurações da Campanha.

Essa separação evita que a tela inicial fique confusa e mantém o contexto correto de trabalho.

RN-011 — Projetos de Jogo podem estar vinculados a um mundo/cenário

Um Projeto de Jogo pode estar vinculado a um Mundo/Cenário.

Exemplo:

Mundo: Aelthos

Projetos de Jogo vinculados:

- A Dama de Ferro;
- A Mina Sem Nome;
- Guerra das Agulhas.

O Mundo/Cenário guarda lore persistente.

O Projeto de Jogo guarda o estado da mesa em andamento.

RN-012 — Mundos/Cenários podem ser reutilizados em múltiplos Projetos de Jogo

Um Mundo/Cenário não deve pertencer obrigatoriamente a uma única campanha.

O mesmo mundo pode ser usado em várias campanhas, crônicas, one-shots, mini-campanhas ou aventuras.

Isso permite preservar mundos vivos ao longo do tempo, mesmo que campanhas diferentes aconteçam em épocas, regiões ou grupos diferentes.

RN-013 — Mundo/Cenário é diferente de Campanha/Crônica

O Nat 1 deve separar claramente Mundo/Cenário de Projeto de Jogo.

Mundo/Cenário representa o universo persistente:

- cosmologia;
- história;
- regiões;
- cidades;
- culturas;
- religiões;
- mitologia;
- povos;
- línguas;
- calendários;
- deuses;
- facções globais;
- eventos históricos;
- mapas amplos;
- lore canônica.

Campanha/Crônica representa a mesa em andamento:

- sessões;
- cenas;
- personagens ativos;
- locais visitados;
- facções envolvidas;
- consequências;
- anotações;
- documentos da campanha;
- estado atual da história.

RN-014 — Projetos de Jogo podem estar vinculados a um sistema

Um Projeto de Jogo pode estar vinculado a um sistema de RPG.

Exemplos:

- D&D 5e;
- Vampiro: A Máscara;
- Tormenta20;
- Ordem Paranormal;
- Call of Cthulhu;
- sistema autoral;
- sistema genérico.

O sistema serve para oferecer regras, templates, fichas, campos sugeridos e compêndios.

O sistema não controla a história da campanha.

RN-015 — Sistemas funcionam como templates editáveis

Sistemas não devem ser estruturas rígidas.

Um sistema deve funcionar como um conjunto de templates e referências editáveis, podendo conter:

- tipos de ficha;
- atributos;
- perícias;

- regras;
- magias;
- poderes;
- itens;
- criaturas;
- condições;
- campos personalizados;
- categorias;
- modelos de personagens;
- modelos de ameaças;
- regras caseiras.

O mestre poderá usar templates prontos, editar templates existentes ou criar sistemas próprios.

RN-016 — O Nat 1 deve suportar sistemas autorais desde o início

O usuário deve poder criar Projetos de Jogo sem depender de sistemas oficiais.

O Nat 1 deve permitir:

- sistema genérico;
- sistema autoral;
- campos personalizados;
- coleções próprias;
- regras próprias;
- tipos de ficha próprios;
- categorias próprias;
- compêndios próprios.

Essa regra garante que o produto não fique preso apenas a RPGs famosos.

RN-017 — O Projeto de Jogo possui módulos internos configuráveis

Cada Projeto de Jogo pode ter um conjunto próprio de módulos ativos.

O mestre poderá ativar, ocultar ou desativar módulos conforme a necessidade da mesa.

Exemplo: uma campanha de investigação pode usar:

- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais;
- Pistas & Segredos;
- Documentos;
- Relações;
- Notas.

Exemplo: uma campanha de fantasia pode usar:

- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais;
- Facções;
- Itens & Equipamentos;
- Magias & Poderes;
- Mapas;
- Regras / Compêndio.

RN-018 — O sistema deve evitar poluição visual

O Nat 1 deve permitir que o mestre mantenha a interface limpa.

Módulos não utilizados devem poder ser ocultados.

A interface deve priorizar clareza, organização e acesso rápido às informações importantes da campanha.

Essa regra é especialmente importante porque o produto será usado durante preparação de sessão e, futuramente, durante a própria narração.

RN-019 — O Nat 1 deve preservar a separação entre conteúdo global e conteúdo de campanha

Conteúdo global pertence ao workspace do mestre.

Exemplos de conteúdo global:

- sistemas;
- templates;
- mundos/cenários;
- documentos globais;
- biblioteca reutilizável;
- notas rápidas globais;
- configurações da conta.

Conteúdo de campanha pertence a um Projeto de Jogo específico.

Exemplos de conteúdo de campanha:

- sessões;
- cenas;
- personagens;
- criaturas;
- locais;
- facções;
- documentos da campanha;
- relações;
- pistas;
- notas;
- handouts;
- estado do mundo vivo da campanha.

RN-020 — O Nat 1 não substitui VTTs

O Nat 1 não tem como objetivo inicial substituir Roll20, Foundry, Owlbear Rodeo ou outros VTTs.

O produto deve funcionar como ferramenta complementar de organização, memória, preparação e consulta da campanha.

O mestre poderá usar o Nat 1 aberto enquanto narra em outro ambiente.

RN-021 — A arquitetura deve evitar estrutura monolítica rígida

O sistema deve ser desenhado para expansão futura.

A arquitetura deve permitir adicionar, em fases posteriores:

- IA contextual;
- RAG;
- importação inteligente de PDFs;
- jogadores;
- permissões;
- visão limitada por personagem;
- modo sessão;
- mapas com pins;
- recaps imersivos;
- exportações;
- chat/mensagens;
- novos temas;
- novos sistemas;
- novos tipos de entidade.

A Fase 1 será simples, mas não deve bloquear essas evoluções.

Parte 2 — Módulos, Entidades, Compêndio, Atlas, Relações e Cânone

RN-022 — O Nat 1 terá módulos amplos e configuráveis

Dentro de um Projeto de Jogo, o Nat 1 deve organizar os conteúdos por módulos amplos, capazes de funcionar em diferentes sistemas de RPG.

Os módulos não devem ser rígidos nem específicos demais para um único sistema.

Exemplos de módulos internos:

- Resumo;
- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Facções;
- Itens & Equipamentos;
- Magias & Poderes;
- Regras / Compêndio;
- Mapas;
- Documentos;
- Handouts;
- Pistas & Segredos;
- Relações;
- Notas;
- Recursos / Soundtracks;
- IA da Campanha;
- Configurações da Campanha.

O mestre poderá ocultar módulos que não utiliza.

RN-023 — Personagens & Criaturas será um módulo amplo

O módulo Personagens & Criaturas deve agrupar seres relevantes para a campanha, mundo ou sistema.

Esse módulo deve comportar:

- Personagens de Jogador;
- NPCs;
- criaturas;
- monstros;
- ameaças naturais;
- ameaças místicas ou sobrenaturais;
- ameaças ambientais;
- aliados;
- vilões;
- figurantes;
- entidades únicas.

O nome “NPCs” não deve ser usado como módulo principal, pois é limitado demais para representar monstros, criaturas, ameaças e entidades.

RN-024 — Personagens & Criaturas deve separar tipo principal, natureza e espécie/linhagem

Cada registro do módulo Personagens & Criaturas deve poder separar três conceitos diferentes:

Tipo principal

Define o papel estrutural do registro dentro da campanha ou sistema.

Exemplos:

- PJ;
- NPC;

- Criatura / Monstro;
- Ameaça natural;
- Ameaça mística / sobrenatural;
- Ameaça ambiental;
- Entidade única;
- Outro / Personalizado.

Natureza / Classificação

Define a categoria fundamental ou mecânica conforme o sistema/template.

Exemplos:

- Humanoide;
- Dragão;
- Morto-vivo;
- Espírito;
- Cainita;
- Lupino;
- Constructo;
- Entidade paranormal;
- Alienígena;
- Outro / Personalizado.

Espécie / Raça / Povo / Linhagem

Define o povo, raça, espécie ou linhagem específica.

Exemplos:

- Humano;
- Elfo;
- Anão;
- Orc;
- Skarn;
- Interrano;
- K'tharriano;
- Brujah;
- Nosferatu;
- Garou;
- Outro / Personalizado.

Esses campos devem ser separados para evitar confusão entre função, natureza mecânica e identidade biológica/cultural/ancestral.

RN-025 — Personagens & Criaturas deve suportar classificações adaptáveis por sistema

As classificações de Personagens & Criaturas devem ser adaptáveis ao sistema/template.

Exemplo em D&D:

- Aberração;
- Besta;
- Celestial;
- Constructo;
- Dragão;
- Elemental;
- Feérico;
- Infernal;
- Gigante;
- Humanoide;
- Monstruosidade;
- Gosma;
- Planta;
- Morto-vivo.

Exemplo em Vampiro:

- Cainita;
- Mortal;
- Ghoul;
- Lupino;
- Espírito;
- Sangue-fraco;
- Caçador.

Exemplo em Ordem Paranormal:

- Humano;
- Paranormal;
- Criatura de Sangue;
- Criatura de Morte;
- Criatura de Conhecimento;
- Criatura de Energia;
- Anomalia.

Exemplo em sistema autoral:

- Skarn;
- K'tharriano;
- Eco;
- Interrano;
- Avatar da Agulha;
- Máquina Senciente.

O Nat 1 não deve forçar classificações de um sistema em outro. O vocabulário deve vir do sistema/template ou ser definido pelo mestre.

RN-026 — PJs e NPCs devem poder coexistir no mesmo módulo

Personagens de Jogador e NPCs podem pertencer ao mesmo módulo, mas devem ter tipo e tratamento próprios.

No MVP, o mestre poderá cadastrar PJs manualmente.

Em fases futuras, jogadores poderão ser vinculados a seus próprios personagens, com permissões, ficha e visão limitada.

RN-027 — Criaturas, monstros e ameaças devem ser catalogáveis

Criaturas, monstros e ameaças devem possuir campos que facilitem consulta, filtragem e futura importação de compêndios.

Campos possíveis:

- nome;
- tipo principal;
- natureza/classificação;
- espécie/raça/povo/linhagem, quando aplicável;
- sistema de origem;
- nível / desafio / categoria de ameaça;
- tamanho;
- habitat;
- comportamento;
- descrição;
- estatísticas;
- habilidades;
- ações;
- fraquezas;
- resistências;
- fonte;
- página;
- status de cânone;
- visibilidade.

Os campos exatos podem variar conforme o sistema/template.

RN-028 — O módulo Locais deve funcionar como Atlas

O módulo Locais deve ser tratado como um Atlas da campanha ou mundo.

Ele deve permitir organizar lugares em hierarquia.

Exemplos:

- mundo;
- continente;
- região;
- reino;
- cidade;
- bairro;
- construção;
- sala;
- dungeon;
- plano;
- dimensão;
- nave;
- estação espacial;
- local personalizado.

Um local pode conter outros locais.

Exemplo:

Mundo: Aelthos

Região: Floresta do Luto

Local: Mina Antiga

RN-029 — Locais devem poder se relacionar com mapas

Um local pode ter um ou mais mapas associados.

Mapas podem representar:

- mundo;
- região;
- cidade;
- dungeon;
- construção;
- batalha;
- rota;
- nave;
- território;
- plano;
- investigação.

No MVP, o mapa pode ser apenas um arquivo/imagem associado.

Em fases futuras, poderá ter pins, camadas, regiões clicáveis e geração interna.

RN-030 — Locais devem se relacionar com entidades do mundo vivo

Um local deve poder se relacionar com:

- personagens;
- criaturas;
- facções;
- itens;
- cenas;
- sessões;

- documentos;
- eventos;
- pistas;
- mapas;
- regras locais;
- perigos.

Exemplos:

- uma criatura vive em uma floresta;
- uma facção controla uma cidade;
- uma cena acontece em uma taverna;
- um mapa pertence a uma dungeon;
- um item está escondido em uma ruína.

RN-031 — Facções são módulo próprio

Facções não devem ser tratadas como pastas dentro de Personagens & Criaturas.

Uma facção representa um grupo, organização, culto, clã, seita, casa, guilda, nação, ordem ou força coletiva.

Exemplos:

- Camarilla;
- Anarchs;
- Ordem do Grande Olho;
- Consórcios;
- culto cósmico;
- casa nobre;
- guilda;
- governo;
- corporação;
- facção alienígena.

Personagens e criaturas podem se relacionar com facções, mas a facção é uma entidade própria.

RN-032 — Itens & Equipamentos podem pertencer ao sistema, mundo ou campanha

Itens e equipamentos podem ser conteúdo de sistema, conteúdo homebrew ou conteúdo específico de campanha.

Exemplos:

- arma comum;
- armadura;
- item mágico;
- relíquia;
- artefato;
- veículo;
- ferramenta;
- consumível;
- documento físico;
- tecnologia;
- item autoral.

Um item pode ser:

- parte do compêndio de sistema;
- criação homebrew do mestre;
- item canônico de um mundo;
- item específico de uma campanha;
- item extraído de um documento.

RN-033 — Magias & Poderes devem ser ligados ao sistema/template

Magias, poderes, rituais, disciplinas, habilidades especiais e técnicas devem estar ligados principalmente ao sistema ou template.

Exemplos:

- magias de D&D;
- disciplinas de Vampiro;
- rituais de Ordem Paranormal;
- magias de Tormenta20;
- poderes psíquicos;
- habilidades de classe;
- poderes homebrew.

O mestre poderá criar novos poderes e magias, editar campos e vinculá-los a campanhas ou mundos específicos.

RN-034 — Regras / Compêndio centraliza referências mecânicas

O módulo Regras / Compêndio deve reunir conteúdos de consulta mecânica.

Pode conter:

- regras base;
- regras opcionais;
- regras caseiras;
- condições;
- tabelas;
- magias;
- poderes;
- itens;
- criaturas;
- classes;
- espécies / povos;
- perícias;
- talentos;
- equipamentos;
- conteúdo importado de documentos.

Esse módulo será importante para futuras consultas por IA e importação inteligente de manuais.

RN-035 — Conteúdos de sistema e conteúdos de campanha devem ser diferenciados

O Nat 1 deve diferenciar:

- 1. conteúdo do sistema;**
- 2. conteúdo homebrew global;**
- 3. conteúdo do mundo/cenário;**
- 4. conteúdo específico da campanha.**

Exemplo:

- “Bola de Fogo” pode ser uma magia do sistema;
- “Chama de Idonmor” pode ser uma magia homebrew do mundo Aelthos;
- “Ritual usado por Marcus” pode ser conteúdo específico de uma campanha.

Essa separação evita misturar regra oficial, criação do mestre e estado narrativo da campanha.

RN-036 — Documentos funcionam como fonte bruta

Documentos enviados pelo mestre devem ser tratados como fontes brutas.

Podem ser:

- PDFs;
- textos;
- Markdown;
- imagens;
- mapas;
- manuais;
- aventuras;
- crônicas;
- cenários;
- livros de monstros;
- listas de magias;
- compêndios de itens;
- regras caseiras;
- resumos de sessão;
- materiais criados com IA.

Documentos não devem ser confundidos com entidades estruturadas.

RN-037 — Documentos podem originar conteúdo estruturado

Em fases futuras, documentos poderão ser analisados e gerar sugestões de conteúdo estruturado.

Exemplos:

- um livro de monstros pode sugerir criaturas;
- um manual de sistema pode sugerir regras e magias;
- um cenário pode sugerir cidades, povos, deuses e culturas;
- uma crônica em PDF pode sugerir NPCs, eventos, locais e relações;
- um suplemento caseiro pode sugerir itens, poderes e regras homebrew.

Nada extraído de documento deve se tornar cânone automaticamente.

RN-038 — Todo conteúdo importado ou gerado deve passar por aprovação do mestre

Conteúdos criados, extraídos ou estruturados por IA devem começar como:

- sugestão;
- rascunho;
- pendente de revisão.

O mestre decide se o conteúdo será:

- canônico;
- não canônico;
- arquivado;
- editado;
- descartado.

A IA sugere.

O mestre aprova.

RN-039 — Cânone deve ser status transversal

O status de cânone pode existir em diferentes tipos de conteúdo.

Exemplos:

- personagem;
- criatura;
- local;

- facção;
- item;
- magia;
- regra;
- cena;
- sessão;
- documento;
- pista;
- nota;
- relação.

Estados iniciais sugeridos:

- canônico;
- rascunho;
- sugestão da IA;
- pendente de revisão;
- não canônico;
- arquivado.

RN-040 — Visibilidade deve ser preparada desde o início

Mesmo sem jogadores no MVP 1, o sistema deve nascer preparado para visibilidade futura.

Estados iniciais possíveis:

- privado do mestre;
- visível futuramente aos jogadores;
- revelado;
- oculto;
- pendente de liberação.

Em fases futuras, a visibilidade poderá ser definida por:

- todos os jogadores;
- jogador específico;
- personagem específico;
- grupo;
- campanha;
- mundo.

RN-041 — Relações são base estrutural do mundo vivo

O Nat 1 deve permitir relações entre conteúdos.

Exemplos:

- personagem pertence a facção;
- criatura vive em local;
- item está com personagem;
- cena acontece em local;
- sessão usou cena;
- mapa representa local;
- documento originou criatura;
- regra pertence a sistema;
- poder pertence a sistema;
- pista aponta para segredo;
- facção controla região;
- evento alterou relação.

Essas relações serão fundamentais para busca, organização e futura IA contextual.

RN-042 — Pastas e coleções personalizadas organizam, mas não substituem relações

O mestre poderá criar pastas e coleções personalizadas dentro dos módulos.

Exemplos:

- Relíquias Perdidas;
- TRAJAX;
- Agulhas Primordiais;
- Contratos Vampíricos;
- Profecias;
- Monstros da Floresta;
- Facções Urbanas.

Essas coleções ajudam na organização visual e mental, mas a estrutura principal do Nat 1 deve continuar relacional.

Um item pode estar em uma coleção e também se relacionar com facções, locais, documentos, personagens e eventos.

RN-043 — O sistema deve permitir homebrew e edição manual

O mestre deve poder criar, editar, duplicar ou adaptar conteúdos.

Isso vale para:

- regras;
- magias;
- poderes;
- itens;
- criaturas;
- personagens;
- locais;
- facções;
- templates;
- campos personalizados;
- coleções;
- relações.

O Nat 1 não deve depender apenas de APIs externas ou conteúdos oficiais.

RN-044 — APIs externas são opcionais, não centrais

O Nat 1 poderá usar APIs externas no futuro quando forem úteis e legalmente adequadas.

Porém, o produto não deve depender delas como mecanismo principal.

O diferencial do Nat 1 será permitir que o mestre organize materiais próprios, documentos, PDFs, homebrews e conteúdos de campanha.

RN-045 — O módulo de mapas poderá evoluir para criação interna de mapas

No MVP, mapas podem ser arquivos ou imagens anexadas e relacionadas a locais.

Em fases futuras, o Nat 1 poderá oferecer:

- pins;
- camadas;
- regiões clicáveis;
- rotas;
- áreas de influência;
- mapas reveláveis;
- criação interna de mapas;
- mapas gerados ou auxiliados por IA.

Essa funcionalidade deve ser planejada como evolução, não como requisito do MVP 1.

Parte 3 — Documentos, Importação, IA, Aprovação e Conteúdo Estruturado

RN-046 — Documentos são fontes de informação

No Nat 1, documentos devem funcionar como fontes brutas de informação.

Um documento pode conter regras, lore, aventura, crônica, manual, cenário, lista de criaturas, magias, itens, mapas, handouts, notas de sessão ou material criado pelo mestre.

Tipos iniciais de documentos:

- PDF;
- TXT;
- Markdown;
- imagem;
- mapa;
- documento de campanha;
- manual de sistema;
- suplemento caseiro;
- aventura;
- crônica;
- cenário;
- resumo de sessão;
- material gerado com IA;
- documento homebrew.

RN-047 — Documentos podem pertencer a contextos diferentes

Um documento pode estar associado a diferentes níveis do Nat 1:

- Workspace do Mestre;
- Sistema;
- Mundo/Cenário;
- Campanha/Crônica;
- Sessão;
- Cena;
- Local;
- Personagem & Criatura;
- Facção;
- Item;
- Regra/Compêndio.

Exemplos:

- um manual de sistema pertence ao Sistema;
- um livro de cenário pertence ao Mundo/Cenário;
- uma crônica em PDF pertence à Campanha/Crônica;
- um handout pertence a uma Cena ou Sessão;
- um mapa pertence a um Local ou Atlas;
- um resumo de sessão pertence a uma Sessão.

RN-048 — O Nat 1 deve permitir upload de documentos próprios do mestre

O sistema não deve impedir o mestre de enviar materiais próprios.

O mestre poderá enviar:

- crônicas escritas por ele;
- aventuras autorais;
- cenários próprios;
- regras caseiras;
- listas de criaturas;
- listas de magias;

- mapas criados por ele;
- PDFs gerados com auxílio de IA;
- anotações pessoais;
- suplementos caseiros.

O Nat 1 deve tratar esses materiais como conteúdo privado do usuário, respeitando revisão, organização e aprovação antes de transformar qualquer parte em conteúdo estruturado.

RN-049 — Documentos importados não viram conteúdo estruturado automaticamente

O upload de um documento não deve criar automaticamente conteúdo canônico.

Quando um documento for analisado no futuro, o sistema poderá sugerir entidades, regras, itens, magias, criaturas, locais, eventos e relações.

Essas sugestões devem entrar como:

- sugestão;
- rascunho;
- pendente de revisão.

O mestre decide o que será aprovado.

RN-050 — A IA deve sugerir, não decidir

A IA do Nat 1 não deve ter autoridade para definir o cânone da campanha ou do mundo.

A IA pode:

- resumir;
- classificar;
- sugerir;
- extrair;
- relacionar;
- comparar;
- apontar possíveis contradições;
- gerar rascunhos;
- preencher campos preliminares.

Mas o mestre decide:

- aprovar;
- editar;
- descartar;
- marcar como não canônico;
- arquivar;
- salvar como rascunho;
- transformar em cânone.

RN-051 — Toda importação inteligente deve ter tela de revisão

Quando o Nat 1 analisar um documento e sugerir conteúdo estruturado, o mestre deve visualizar uma tela de revisão antes de salvar.

Essa tela deve permitir:

- ver o conteúdo detectado;
- editar campos;
- alterar tipo;
- alterar natureza/classificação;
- alterar espécie/raça/povo/linhagem, quando aplicável;
- alterar fonte;
- alterar status;
- aprovar individualmente;
- aprovar em lote;
- descartar individualmente;

- descartar em lote.

Exemplo:

Documento: Livro de Criaturas.pdf

Sugestões detectadas:

- Dragão Vermelho — Criatura / Monstro;
- Esqueleto — Criatura / Morto-vivo;
- Lobo Atroz — Ameaça natural;
- Fênix Sombria — Criatura mística.

Cada item deve poder ser revisado antes de entrar no compêndio.

RN-052 — Conteúdos extraídos devem preservar origem e fonte

Todo conteúdo extraído de documento deve manter referência à sua origem.

Campos importantes:

- documento de origem;
- página, quando disponível;
- trecho ou seção de origem;
- data de importação;
- usuário que importou;
- status de revisão;
- status de cânone;
- contexto de destino.

Isso permite rastreabilidade e confiança.

Exemplo:

Criatura: Dragão Vermelho

Fonte: Livro de Criaturas.pdf

Página: 87

Status: pendente de revisão

Destino sugerido: Sistema > Compêndio > Criaturas

RN-053 — O Nat 1 deve diferenciar conteúdo oficial, homebrew e conteúdo da campanha

O sistema deve permitir classificar a origem do conteúdo.

Origens possíveis:

- sistema oficial;
- SRD / conteúdo aberto;
- homebrew global;
- homebrew da campanha;
- conteúdo autoral do mestre;
- documento importado;
- sugestão da IA;
- criado manualmente;
- material de sessão.

Essa separação evita misturar regra de sistema, adaptação do mestre e conteúdo narrativo da campanha.

RN-054 — A importação de manuais deve respeitar o sistema/template

Quando o mestre importar um manual, o Nat 1 deve considerar o sistema/template associado.

Exemplo:

Em D&D, uma criatura pode ter natureza como dragão, besta, morto-vivo ou aberração.

Em Vampiro, as classificações podem ser Cainita, mortal, ghoul, lupino, espírito ou caçador.

Em um sistema autoral, o mestre pode definir naturezas próprias.

O Nat 1 não deve forçar classificações de um sistema em outro.

RN-055 — O Nat 1 deve permitir importação em massa futura

Em fases futuras, o sistema poderá importar em massa conteúdos de documentos.

Exemplos:

- criaturas de um livro de monstros;
- magias de um manual;
- itens de um compêndio;
- regiões de um cenário;
- deuses de um panteão;
- facções de uma crônica;
- NPCs de uma aventura;
- eventos de uma linha do tempo;
- regras caseiras de um suplemento.

A importação em massa deve sempre gerar uma etapa de revisão.

RN-056 — O Nat 1 deve permitir edição manual de qualquer conteúdo importado

Conteúdo importado não deve ficar bloqueado.

O mestre deve poder:

- corrigir nome;
- corrigir categoria;
- alterar natureza;
- alterar espécie/raça/povo/linhagem, quando aplicável;
- editar descrição;
- adicionar notas;
- remover informações;
- vincular a locais;
- vincular a facções;
- ajustar status de cânone;
- ajustar visibilidade;
- marcar como homebrew;
- duplicar e adaptar.

RN-057 — O Nat 1 deve permitir criação manual mesmo sem documentos

A importação de documentos é um diferencial, mas não deve ser obrigatória.

O mestre deve conseguir usar o sistema manualmente, criando:

- campanhas;
- mundos;
- sistemas;
- sessões;
- cenas;
- personagens;
- criaturas;
- locais;
- facções;
- itens;
- magias;
- regras;
- mapas;
- documentos;
- notas;
- relações.

O Nat 1 deve funcionar bem mesmo antes das fases de IA e RAG.

RN-058 — A busca deve evoluir em camadas

No MVP 1, a busca pode ser simples, baseada em texto, filtros e categorias.

Em fases futuras, a busca deve evoluir para:

- busca por tags;
- busca por relações;
- busca por tipo de conteúdo;
- busca por documento de origem;
- busca por status de cânone;
- busca por visibilidade;
- busca semântica;
- perguntas à campanha;
- perguntas ao compêndio;
- perguntas a documentos específicos.

RN-059 — A IA deve usar apenas contexto autorizado

Quando a IA for implementada, ela deve respeitar o contexto selecionado.

Exemplos:

- IA da Campanha usa dados da campanha;
- IA do Mundo usa dados do mundo/cenário;
- IA do Sistema usa regras e compêndio;
- IA de Documento usa o documento selecionado;
- IA de Jogador, no futuro, usa apenas conteúdo visível ao jogador/personagem.

A IA não deve misturar campanhas, mundos ou documentos sem autorização/contexto claro.

RN-060 — A IA deve evitar contradizer o cânone aprovado

Ao gerar conteúdo, a IA deve considerar primeiro conteúdos canônicos e aprovados.

Conteúdos em rascunho, sugestão ou pendentes de revisão podem ser usados apenas se o mestre permitir.

A IA deve ser capaz de alertar quando uma sugestão parece contradizer:

- evento canônico;
- status de personagem;
- localização;
- relação;
- regra aprovada;
- estado de mundo;
- sessão anterior;
- segredo já definido.

RN-061 — Conteúdos de IA devem ser salvos como rascunho por padrão

Qualquer texto, entidade, regra, cena, resumo, relação ou item gerado pela IA deve nascer como rascunho ou sugestão.

O mestre deve revisar antes de canonizar.

Estados possíveis:

- sugestão da IA;
- rascunho;
- pendente de revisão;
- aprovado / canônico;
- descartado;
- arquivado.

RN-062 — O Nat 1 deve permitir documentos vivos e documentos arquivados

Documentos podem estar ativos ou arquivados.

Um documento ativo pode ser usado para consulta, referência e futuras sugestões.

Um documento arquivado permanece salvo, mas não deve ser usado automaticamente pela IA, salvo se o mestre ativar.

Estados possíveis:

- ativo;
- arquivado;
- pendente de extração;
- extraído;
- com erro de extração;
- pendente de revisão;
- processado.

RN-063 — O Nat 1 deve registrar histórico mínimo de alterações importantes

Conteúdos importantes devem registrar alterações relevantes, principalmente quando envolvem cânone.

Exemplos de alterações rastreáveis:

- rascunho aprovado como canônico;
- conteúdo canônico editado;
- documento importado;
- entidade extraída de documento;
- relação criada;
- relação removida;
- visibilidade alterada;
- conteúdo arquivado;
- item descartado.

No MVP, esse histórico pode ser simples.

Em fases futuras, pode evoluir para versionamento completo.

RN-064 — O Nat 1 deve permitir revisão em lote

Quando muitos conteúdos forem importados de um documento, o sistema deve permitir revisão em lote.

Exemplos:

- aprovar todas as criaturas selecionadas;
- descartar todas as entradas duplicadas;
- marcar grupo como rascunho;
- alterar destino de vários itens;
- aplicar tag em massa;
- mover itens para uma coleção;
- alterar status para pendente de revisão.

Essa regra é essencial para importação de livros, manuais e compêndios.

RN-065 — Conteúdos estruturados devem poder voltar ao documento de origem

Quando uma entidade for criada a partir de um documento, o mestre deve poder acessar sua fonte.

Exemplo:

Criatura: Lobo Atroz

Origem: Livro de Criaturas.pdf, página 34

Ação: “Ver origem”

Essa regra facilita conferência, revisão e confiança na informação extraída.

Parte 4 — Sessões, Cenas, Mundo Vivo e Preparação de Mesa

RN-066 — Sessões representam encontros reais de jogo

Uma Sessão representa um encontro real de jogo, com data, duração, participantes, acontecimentos e consequências.

A Sessão deve servir para registrar o que aconteceu na mesa, e não apenas como uma nota solta.

Uma Sessão pode conter:

- título;
- número ou ordem;
- data real;
- data fictícia, quando aplicável;
- resumo;
- anotações brutas;
- acontecimentos importantes;
- decisões dos jogadores;
- personagens presentes;
- NPCs envolvidos;
- locais visitados;
- cenas jogadas;
- cenas não utilizadas;
- consequências;
- pendências;
- ganchos para a próxima sessão;
- documentos ou handouts usados;
- status da sessão.

RN-067 — Sessões podem ter status

Uma Sessão deve poder ter status para auxiliar a organização do mestre.

Status iniciais sugeridos:

- planejada;
- em preparação;
- agendada;
- jogada;
- resumida;
- revisada;
- cancelada;
- arquivada.

Esses status ajudam o mestre a separar sessões futuras, sessões já jogadas e sessões que ainda precisam de resumo ou revisão.

RN-068 — Sessões devem registrar consequências

O Nat 1 deve permitir que uma Sessão registre consequências narrativas e mecânicas.

Exemplos:

- NPC morreu;
- facção mudou de atitude;
- local foi destruído;
- pista foi revelada;
- segredo foi descoberto;
- item mudou de posse;
- personagem ganhou inimigo;
- relógio narrativo avançou;
- regra caseira foi criada;
- mapa foi atualizado;

- documento foi entregue aos jogadores.

Essas consequências devem poder se relacionar com entidades do mundo vivo.

RN-069 — Sessões devem poder atualizar o estado do mundo vivo

O registro de uma Sessão deve poder alterar ou sugerir alterações no estado do mundo.

Exemplos:

- alterar status de um personagem para morto, desaparecido, ferido ou aliado;
- marcar uma pista como revelada;
- marcar um segredo como descoberto;
- alterar relação entre facções;
- vincular um evento a uma linha do tempo;
- atualizar a posse de um item;
- registrar que uma cena foi jogada;
- registrar que um local foi visitado.

No MVP, essas alterações podem ser manuais.

Em fases futuras, a IA poderá sugerir atualizações a partir do resumo da sessão, sempre com aprovação do mestre.

RN-070 — Cenas representam unidades narrativas

Uma Cena é uma unidade narrativa preparada, improvisada ou registrada dentro de uma Campanha/Crônica.

Uma Cena pode representar:

- encontro social;
- combate;
- investigação;
- exploração;
- revelação;
- perseguição;
- negociação;
- ritual;
- descanso;
- transição;
- dilema moral;
- confronto final.

A Cena deve ajudar o mestre a preparar e organizar a sessão sem obrigar uma estrutura rígida.

RN-071 — Cenas podem estar ligadas a sessões, arcos ou campanhas

Uma Cena pode estar vinculada a:

- uma Sessão específica;
- um Arco;
- uma Campanha/Crônica;
- um Local;
- um Mapa;
- uma Facção;
- um Documento;
- uma Pista;
- um Evento;
- uma ou mais Personagens & Criaturas.

Isso permite que cenas preparadas sejam usadas, movidas, reaproveitadas ou arquivadas.

RN-072 — Cenas podem ter status

Uma Cena deve poder ter status próprio.

Status iniciais sugeridos:

- ideia;
- preparada;
- pronta para jogar;
- jogada;
- improvisada;
- pulada;
- descartada;
- reaproveitável;
- arquivada.

Esses status ajudam o mestre a saber quais cenas estão prontas, quais ainda são rascunho e quais podem ser usadas futuramente.

RN-073 — Cenas devem poder conter estrutura interna simples

Uma Cena pode conter campos como:

- título;
- objetivo da cena;
- resumo;
- descrição para o mestre;
- texto para ler aos jogadores;
- personagens envolvidos;
- local;
- mapa;
- pistas presentes;
- conflito;
- possíveis escolhas;
- consequências prováveis;
- testes/desafios;
- recompensas;
- segredos;
- notas privadas.

O mestre não é obrigado a preencher todos esses campos.

O Nat 1 deve permitir uso simples ou detalhado.

RN-074 — Cenas não devem obrigar linearidade

O Nat 1 não deve forçar que todas as campanhas sejam lineares.

Cenas podem ser preparadas sem estarem presas a uma ordem fixa.

Isso é importante para campanhas sandbox, investigação, terror, mundo aberto e crônicas políticas.

O mestre deve poder reorganizar cenas, mover cenas entre sessões ou deixar cenas como banco de preparação.

RN-075 — O Nat 1 deve permitir Arcos narrativos

Um Arco é uma divisão narrativa maior dentro de uma Campanha/Crônica.

Exemplos:

- Batismo de Sangue;
- O Cozinheiro;
- Sistema Circulatório;
- A Dama de Ferro;
- A Guerra das Agulhas;
- O Caso do Farol;
- A Queda de Valerius.

Um Arco pode conter:

- sessões;
- cenas;

- eventos;
- personagens envolvidos;
- facções envolvidas;
- locais principais;
- documentos;
- consequências;
- status.

No MVP, Arcos podem ser opcionais ou representados de forma simples.

Em fases futuras, podem ganhar tela própria.

RN-076 — Mundo Vivo representa o estado atual da campanha

O Mundo Vivo é o conjunto de informações que representa o estado atual de uma Campanha/Crônica ou Mundo/Cenário.

Ele inclui:

- quem está vivo, morto, desaparecido ou ativo;
- quais facções ganharam ou perderam poder;
- quais locais foram alterados;
- quais segredos foram revelados;
- quais pistas estão abertas;
- quais eventos aconteceram;
- quais relações mudaram;
- quais documentos foram descobertos;
- quais itens mudaram de posse;
- quais consequências ainda estão pendentes.

O Mundo Vivo deve ser construído a partir de entidades, sessões, cenas, relações e eventos.

RN-077 — O Nat 1 deve preservar histórico de acontecimentos importantes

A Campanha/Crônica deve registrar acontecimentos importantes ao longo do tempo.

Esses acontecimentos podem vir de:

- sessões;
- cenas;
- decisões dos jogadores;
- mudanças em relações;
- alterações de status;
- eventos criados pelo mestre;
- importação de documentos;
- sugestões aprovadas da IA.

Esse histórico será essencial para consultas futuras, resumos, recaps e IA contextual.

RN-078 — Eventos representam acontecimentos relevantes

Um Evento representa algo que aconteceu ou está planejado dentro do mundo, campanha ou sessão.

Exemplos:

- morte de um NPC;
- queda de uma cidade;
- início de uma guerra;
- revelação de um segredo;
- roubo de um item;
- avanço de uma facção;
- ritual concluído;
- descoberta de uma pista;
- desaparecimento de um personagem;
- mudança política;
- consequência de uma sessão.

Eventos podem estar ligados a sessões, cenas, locais, personagens, facções, documentos e linha do tempo.

RN-079 — Eventos podem ter escopo

Um Evento pode ter escopo diferente.

Escopos possíveis:

- pessoal;
- grupo;
- local;
- regional;
- mundial;
- cósmico;
- político;
- sobrenatural;
- mecânico;
- narrativo.

Isso ajuda o mestre e a IA futura a entenderem o impacto do acontecimento.

RN-080 — Relações devem registrar tipo e contexto

Uma relação entre conteúdos deve registrar mais do que apenas “A está ligado a B”.

Uma relação deve poder conter:

- origem;
- destino;
- tipo;
- descrição;
- intensidade;
- status;
- visibilidade;
- fonte;
- data ou sessão de origem;
- notas do mestre.

Exemplos de tipo de relação:

- pertence a;
- controla;
- rivaliza com;
- serve a;
- protege;
- odeia;
- conhece;
- investiga;
- ameaça;
- vive em;
- ocorreu em;
- possui;
- criou;
- destruiu;
- revelou;
- esconde;
- depende de;
- foi extraído de.

RN-081 — Relações podem mudar com o tempo

Relações não devem ser tratadas como informações estáticas.

Uma relação pode mudar conforme a campanha avança.

Exemplos:

- aliado vira inimigo;
- facção perde controle de um local;
- NPC deixa de servir uma organização;
- segredo revelado muda confiança entre personagens;
- item muda de dono;
- cidade deixa de ser segura;
- criatura passa a caçar os jogadores.

O Nat 1 deve permitir edição e, futuramente, histórico dessas mudanças.

RN-082 — Relações ajudam a IA a manter coerência

As relações entre conteúdos serão uma base importante para IA contextual futura.

A IA deve poder usar relações para responder perguntas como:

- quem controla este local?
- que NPCs pertencem a esta facção?
- quais criaturas vivem neste mapa?
- que pistas apontam para este segredo?
- que eventos envolveram este personagem?
- quais documentos originaram esta regra?
- quem sabe sobre este acontecimento?

Essa regra reforça que relações são parte central da arquitetura do Nat 1.

RN-083 — O Nat 1 deve permitir preparação de sessão

O sistema deve oferecer recursos para o mestre preparar uma sessão.

A preparação de sessão pode incluir:

- escolher cenas;
- selecionar NPCs importantes;
- selecionar locais;
- anexar mapas;
- anexar handouts;
- listar pistas;
- listar segredos;
- registrar possíveis consequências;
- anotar lembretes;
- definir objetivo da sessão;
- registrar tarefas pendentes.

No MVP, isso pode ser feito dentro do módulo Sessões e Cenas.

Em fases futuras, pode existir um Modo Preparação.

RN-084 — O Nat 1 deve permitir notas rápidas durante preparação

O mestre deve poder registrar notas rápidas sem precisar decidir imediatamente onde elas pertencem.

Uma nota rápida pode depois ser convertida ou relacionada a:

- sessão;
- cena;
- personagem;
- local;
- facção;
- pista;
- segredo;
- item;

- documento;
- evento.

Isso evita perda de ideias e melhora a fluidez de preparação.

RN-085 — Notas podem ser globais ou de campanha

Notas podem existir em contextos diferentes:

- nota global do mestre;
- nota de mundo/cenário;
- nota de campanha/crônica;
- nota de sessão;
- nota de cena;
- nota vinculada a entidade.

O sistema deve preservar o contexto da nota para evitar confusão.

RN-086 — O Nat 1 deve diferenciar notas simples de conteúdo estruturado

Nem toda informação precisa virar entidade estruturada.

O mestre pode querer apenas escrever uma nota simples.

O Nat 1 deve permitir:

- escrita rápida;
- organização mínima;
- posterior conversão em conteúdo estruturado;
- vinculação a entidades;
- arquivamento.

Isso evita obrigar o mestre a preencher muitos formulários para ideias pequenas.

RN-087 — O módulo de Pistas & Segredos deve apoiar campanhas investigativas

O Nat 1 deve oferecer suporte a pistas e segredos, especialmente para campanhas de investigação, terror, Vampiro, Ordem Paranormal e Call of Cthulhu.

Uma pista pode ter status como:

- não revelada;
- revelada parcialmente;
- revelada;
- interpretada incorretamente;
- falsa;
- descartada.

Um segredo pode ter status como:

- oculto;
- pressentido;
- parcialmente revelado;
- revelado;
- conhecido por personagens específicos.

RN-088 — Pistas e segredos devem poder se relacionar

Pistas e segredos devem poder ser conectados.

Exemplos:

- pista aponta para segredo;
- pista falsa desvia de segredo;
- documento contém pista;
- NPC sabe segredo;
- local guarda pista;
- item revela segredo;

- facção esconde segredo.

Essa estrutura será essencial para investigações e campanhas políticas.

RN-089 — O Nat 1 deve permitir pendências de campanha

O sistema deve permitir registrar pendências abertas.

Exemplos:

- NPC precisa reagir;
- facção precisa agir;
- pista ainda não foi explorada;
- cena foi pulada;
- item precisa ter consequência;
- jogador pediu informação;
- documento precisa ser revisado;
- IA sugeriu conteúdo pendente;
- relação precisa ser atualizada.

Pendências podem aparecer no Dashboard da Campanha.

RN-090 — O Nat 1 deve permitir lembretes

O mestre deve poder criar lembretes associados a:

- campanha;
- sessão;
- cena;
- personagem;
- documento;
- evento;
- tarefa geral.

No MVP, lembretes podem ser simples.

Em fases futuras, podem se integrar a calendário, notificações e agenda.

RN-091 — O Dashboard da Campanha deve destacar o que importa para a próxima sessão

O Dashboard da Campanha deve ajudar o mestre a retomar rapidamente o contexto.

Ele pode destacar:

- próxima sessão;
- última sessão jogada;
- cenas preparadas;
- pendências;
- NPCs relevantes;
- locais recentes;
- documentos recentes;
- notas rápidas;
- pistas abertas;
- consequências pendentes;
- busca da campanha.

O objetivo é reduzir o tempo que o mestre perde tentando lembrar o estado da campanha.

RN-092 — O sistema deve suportar uso simples e uso avançado

O Nat 1 deve ser útil tanto para mestres que querem apenas registrar notas simples quanto para mestres que desejam uma estrutura detalhada.

O mestre não deve ser obrigado a preencher todos os campos.

O sistema deve permitir:

- uso simples com notas e listas;

- uso intermediário com entidades e sessões;
- uso avançado com relações, mundo vivo, documentos, mapas, cânone e visibilidade.

Essa regra é essencial para evitar que o produto pareça pesado demais.

Requisitos Funcionais do MVP 1

Parte 1 — Acesso, Home do Mestre, Campanhas & Crônicas e Dashboard

1. Escopo desta seção

Esta seção define os requisitos funcionais iniciais do MVP 1 relacionados a:

- autenticação;
- acesso do usuário;
- Home do Mestre;
- listagem de Campanhas & Crônicas;
- criação e edição de Projeto de Jogo;
- Dashboard do Projeto de Jogo;
- estados vazios, validações e feedbacks básicos.

O MVP 1 é uma versão organizacional focada no mestre/narrador.

Não fazem parte desta primeira versão:

- IA pesada;
- RAG;
- upload inteligente com extração automática;
- jogadores;
- chat;
- mapas interativos com pins;
- recaps visuais;
- exportações complexas;
- soundtracks funcionais;
- fichas completas por sistema.

2. Acesso e autenticação

RF-001 — O usuário deve poder criar uma conta

O sistema deve permitir que um usuário crie uma conta informando, no mínimo:

- nome;
- e-mail;
- senha.

Após o cadastro bem-sucedido, o usuário deve poder acessar a Home do Mestre.

Critérios de aceite

- O sistema não deve permitir cadastro sem nome.
- O sistema não deve permitir cadastro sem e-mail.
- O sistema não deve permitir cadastro sem senha.
- O sistema deve validar formato de e-mail.
- O sistema deve impedir cadastro duplicado com o mesmo e-mail.
- O sistema deve exibir mensagem clara em caso de erro.

RF-002 — O usuário deve poder fazer login

O sistema deve permitir login usando e-mail e senha.

Após autenticação bem-sucedida, o usuário deve ser redirecionado para a Home do Mestre.

Critérios de aceite

- O sistema deve validar e-mail e senha.
- O sistema deve exibir erro quando as credenciais forem inválidas.

- O sistema deve impedir acesso interno sem autenticação.
- Após login válido, o usuário deve acessar a Home do Mestre.

RF-003 — O usuário deve poder sair da conta

O sistema deve permitir logout.

Após sair da conta, o usuário deve ser redirecionado para a tela de login ou tela pública inicial.

Critérios de aceite

- O usuário autenticado deve ter acesso a uma ação de logout.
- Após logout, rotas protegidas não devem permanecer acessíveis.
- O token/sessão local deve ser removido ou invalidado no cliente.

RF-004 — O sistema deve proteger rotas internas

Telas internas devem ser acessíveis apenas para usuários autenticados.

Rotas protegidas no MVP 1:

- Home do Mestre;
- Campanhas & Crônicas;
- criação de Projeto de Jogo;
- Dashboard do Projeto de Jogo;
- módulos internos do Projeto de Jogo;
- configurações internas.

Critérios de aceite

- Usuário não autenticado tentando acessar rota interna deve ser redirecionado para login.
- Usuário autenticado deve conseguir navegar pelas rotas internas permitidas.
- O sistema não deve exibir dados privados sem autenticação.

RF-005 — O sistema deve associar dados ao usuário autenticado

Todos os dados criados no MVP 1 devem pertencer ao usuário autenticado.

Dados vinculados ao usuário:

- Projetos de Jogo;
- mundos/cenários;
- notas globais;
- documentos cadastrados;
- configurações;
- sistemas/templates criados ou personalizados.

No MVP 1, não haverá compartilhamento entre usuários.

Critérios de aceite

- Um usuário não deve visualizar dados de outro usuário.
- Um Projeto de Jogo deve estar vinculado ao usuário dono.
- Consultas de listagem devem retornar apenas dados do usuário autenticado.

3. Home do Mestre

RF-006 — O usuário deve acessar a Home do Mestre após login

Após login, o sistema deve exibir a Home do Mestre como ponto de entrada global do usuário.

A Home do Mestre representa o workspace geral do mestre.

Critérios de aceite

- Após login válido, o usuário deve ser redirecionado para a Home.
- A Home deve carregar dados pertencentes ao usuário autenticado.
- A Home deve funcionar mesmo quando o usuário ainda não possui campanhas.

RF-007 — A Home do Mestre deve exibir estado vazio quando não houver Campanhas & Crônicas

Se o usuário ainda não criou nenhum Projeto de Jogo, a Home deve exibir um estado vazio útil.

O estado vazio deve orientar o usuário a criar sua primeira campanha/crônica.

Critérios de aceite

- Usuário sem campanhas deve ver mensagem clara.
- Usuário sem campanhas deve ver ação para criar novo Projeto de Jogo.
- A tela não deve parecer quebrada ou incompleta.

RF-008 — A Home do Mestre deve exibir a última Campanha/Crônica acessada

Quando existir histórico de acesso, a Home deve exibir um card de destaque para o último Projeto de Jogo acessado.

Esse card pode conter:

- nome;
- formato;
- sistema/template;
- tema visual;
- última atualização;
- botão para continuar.

Critérios de aceite

- Se houver último Projeto acessado, ele deve aparecer em destaque.
- O botão de continuar deve abrir o Dashboard do Projeto de Jogo.
- Se não houver último Projeto acessado, a Home deve exibir estado alternativo adequado.

RF-009 — A Home do Mestre deve listar Campanhas & Crônicas recentes

A Home deve exibir uma lista ou grade com Projetos de Jogo recentes do usuário.

Cada card deve exibir informações básicas, quando disponíveis:

- nome;
- formato;
- sistema/template;
- status;
- tema visual;
- última atualização.

Indicadores como quantidade de sessões, personagens ou documentos podem aparecer quando já estiverem disponíveis.

Critérios de aceite

- A Home deve listar apenas Projetos do usuário autenticado.
- Cada card deve permitir abrir o respectivo Dashboard.
- A listagem deve funcionar com zero, um ou múltiplos Projetos.

RF-010 — A Home do Mestre deve permitir criar novo Projeto de Jogo

A Home deve conter uma ação clara para criar um novo Projeto de Jogo.

Nome sugerido para a ação:

- Nova campanha/crônica;
- Criar campanha;

- Novo Projeto de Jogo.

A nomenclatura final poderá ser refinada na etapa de UX, mas a ação deve ser evidente.

Critérios de aceite

- A ação deve estar visível na Home.
- Ao clicar, o usuário deve ser levado para o fluxo de criação.
- O fluxo deve estar disponível para usuário autenticado.

RF-011 — A Home do Mestre deve ter navegação global simples

A Home deve apresentar navegação global com poucos itens principais.

Itens obrigatórios no MVP 1:

- Home;
- Campanhas & Crônicas;
- Configurações.

Itens previstos estruturalmente, podendo entrar como versão simples ou placeholder no MVP 1:

- Mundos / Cenários;
- Sistemas & Templates;
- Biblioteca Global;
- Notas rápidas.

Critérios de aceite

- A navegação global não deve exibir módulos internos de campanha como itens principais.
- Itens não implementados completamente não devem quebrar a navegação.
- A navegação deve deixar claro que o usuário está no nível global do sistema.

RF-012 — A Home do Mestre deve oferecer busca global mínima

O sistema deve oferecer busca global simples no nível da Home.

No MVP 1, a busca global deve priorizar:

- Campanhas & Crônicas;
- Mundos/Cenários, se implementados;
- notas globais, se implementadas.

Busca em entidades internas da campanha pode existir apenas dentro do contexto da campanha.

Critérios de aceite

- O usuário deve conseguir pesquisar Projetos de Jogo pelo nome.
- Quando não houver resultado, o sistema deve exibir estado vazio de busca.
- A busca global não deve misturar indevidamente dados de campanhas diferentes sem contexto claro.

RF-013 — A Home do Mestre deve permitir notas rápidas globais em versão simples

O usuário deve poder registrar notas rápidas globais.

No MVP 1, notas rápidas podem ser simples, contendo:

- título opcional;
- texto;
- data de criação;
- data de atualização;
- status simples: ativa ou arquivada.

Critérios de aceite

- O usuário deve poder criar uma nota rápida.
- O usuário deve poder editar uma nota rápida.
- O usuário deve poder arquivar uma nota rápida.

- Notas globais devem pertencer ao usuário autenticado.

4. Campanhas & Crônicas / Projetos de Jogo

RF-014 — O usuário deve poder listar seus Projetos de Jogo

O sistema deve permitir visualizar todos os Projetos de Jogo do usuário na seção Campanhas & Crônicas.

A listagem deve exibir:

- nome;
- formato;
- sistema/template;
- status;
- tema visual;
- data de criação;
- data da última atualização.

Campos opcionais, se existirem:

- mundo/cenário vinculado;
- imagem/capa;
- descrição curta.

Critérios de aceite

- A lista deve exibir apenas Projetos do usuário autenticado.
- O usuário deve poder abrir um Projeto da lista.
- A lista deve ter estado vazio quando não houver Projetos.
- A lista deve permitir diferenciar Projetos ativos e arquivados.

RF-015 — O usuário deve poder criar um Projeto de Jogo

O sistema deve permitir criar um novo Projeto de Jogo.

Campos obrigatórios no MVP 1:

- nome;
- formato.

Campos com valor padrão:

- sistema/template: Genérico;
- tema visual: Cartógrafo;
- status: ativo ou em preparação.

Campos opcionais:

- descrição curta;
- mundo/cenário vinculado;
- imagem/capa;
- notas iniciais.

Critérios de aceite

- O usuário deve conseguir criar um Projeto informando apenas nome e formato.
- Se o usuário não escolher sistema/template, o sistema deve usar Genérico.
- Se o usuário não escolher tema, o sistema deve usar Cartógrafo.
- O Projeto criado deve aparecer na lista de Campanhas & Crônicas.
- Após criação, o usuário deve poder acessar o Dashboard do Projeto.

RF-016 — O usuário deve escolher o formato do Projeto de Jogo

Durante a criação, o usuário deve poder escolher o formato do Projeto.

Formatos iniciais:

- Campanha;
- Crônica;
- Caso / Investigação;
- Aventura;
- One-shot;
- Mini-campanha;
- Sandbox;
- Outro / Personalizado.

O formato não deve limitar os módulos do Projeto. Ele serve para organização e adaptação de linguagem.

Critérios de aceite

- O campo formato deve ser obrigatório.
- O usuário deve poder escolher um formato da lista.
- O usuário deve poder escolher Outro/Personalizado, se aplicável.
- O formato escolhido deve ser exibido no Dashboard ou detalhes do Projeto.

RF-017 — O usuário deve poder escolher ou manter sistema/template genérico

Durante a criação ou edição de um Projeto de Jogo, o usuário deve poder escolher um sistema/template.

Opções iniciais recomendadas:

- Genérico;
- Sistema autoral;
- D&D 5e;
- Vampiro: A Máscara;
- Tormenta20;
- Ordem Paranormal;
- Call of Cthulhu;
- Outro.

No MVP 1, esses templates podem ser simples, servindo principalmente para nomenclatura, campos sugeridos e organização inicial.

Critérios de aceite

- O sistema/template deve poder ser Genérico.
- O sistema/template escolhido deve ficar salvo no Projeto.
- O sistema/template não deve impedir edição manual pelo mestre.
- O usuário deve poder alterar o sistema/template posteriormente, com aviso se houver impacto futuro.

RF-018 — O usuário deve poder vincular um Mundo/Cenário ao Projeto de Jogo

Durante a criação ou edição do Projeto, o usuário deve poder:

- não vincular mundo/cenário;
- criar um novo mundo/cenário simples;
- vincular a um mundo/cenário existente.

No MVP 1, esse vínculo deve ser opcional.

Critérios de aceite

- O usuário deve conseguir criar Projeto sem Mundo/Cenário vinculado.
- O usuário deve conseguir vincular um Mundo/Cenário existente, se houver.
- O usuário deve conseguir criar um Mundo/Cenário simples durante ou após a criação do Projeto.
- O vínculo deve aparecer nos detalhes do Projeto, quando existir.

RF-019 — O usuário deve poder escolher tema visual do Projeto de Jogo

Durante a criação ou edição do Projeto, o usuário deve poder escolher um tema visual.

Temas definidos na direção visual:

- Cartógrafo;
- Sombrio/Terror;
- Futurista.

No MVP 1, o tema obrigatório funcional será Cartógrafo.

Sombrio/Terror e Futurista devem estar previstos na estrutura de dados e design tokens, mas podem ser implementados visualmente em versão inicial ou fase posterior.

Critérios de aceite

- Todo Projeto deve ter um tema salvo.
- Se o usuário não escolher tema, o padrão deve ser Cartógrafo.
- O tema escolhido deve ser aplicado ao Dashboard quando implementado.
- A estrutura deve permitir adicionar novos temas sem alterar a lógica principal do Projeto.

RF-020 — O usuário deve poder selecionar módulos ativos do Projeto de Jogo

O usuário deve poder configurar quais módulos aparecem no menu interno do Projeto.

Módulos obrigatórios no MVP 1:

- Resumo;
- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Facções;
- Documentos;
- Notas;
- Relações;
- Configurações da Campanha.

Módulos previstos para fases futuras ou versão simples:

- Itens & Equipamentos;
- Magias & Poderes;
- Regras / Compêndio;
- Mapas;
- Handouts;
- Pistas & Segredos;
- Recursos / Soundtracks;
- IA da Campanha.

Critérios de aceite

- O usuário deve poder ocultar módulos não utilizados.
- Módulos ocultos não devem aparecer como destaque no menu interno.
- O Projeto deve manter salvo o conjunto de módulos ativos.
- Módulos futuros podem aparecer como indisponíveis, planejados ou simplesmente não aparecer no MVP.

RF-021 — O usuário deve poder editar um Projeto de Jogo

O sistema deve permitir editar informações básicas do Projeto.

Campos editáveis:

- nome;
- formato;
- descrição;
- sistema/template;
- mundo/cenário vinculado;
- tema visual;
- status;

- módulos ativos;
- imagem/capa;
- notas gerais.

Critérios de aceite

- Alterações devem ser persistidas.
- O sistema deve exibir feedback de sucesso ou erro ao salvar.
- Campos obrigatórios não devem ser salvos vazios.
- A edição deve respeitar dados do usuário autenticado.

RF-022 — O usuário deve poder arquivar um Projeto de Jogo

O sistema deve permitir arquivar um Projeto de Jogo.

Projetos arquivados não devem aparecer em destaque na Home, mas devem continuar acessíveis por filtro ou seção específica.

Critérios de aceite

- O usuário deve conseguir arquivar um Projeto.
- Projeto arquivado deve deixar de aparecer como Projeto ativo.
- O usuário deve conseguir visualizar Projetos arquivados.
- O sistema deve permitir restaurar Projeto arquivado, se definido no MVP.

5. Dashboard do Projeto de Jogo

RF-023 — O usuário deve poder acessar o Dashboard do Projeto de Jogo

Ao clicar em uma Campanha/Crônica, o usuário deve ser levado para o Dashboard daquele Projeto.

O Dashboard deve carregar o contexto específico do Projeto selecionado.

Critérios de aceite

- O Dashboard deve abrir apenas Projetos pertencentes ao usuário autenticado.
- O Dashboard deve exibir dados do Projeto correto.
- O usuário deve conseguir retornar para a Home ou listagem de Campanhas & Crônicas.

RF-024 — O Dashboard deve exibir resumo do Projeto

O Dashboard deve exibir informações básicas do Projeto:

- nome;
- formato;
- sistema/template;
- mundo/cenário vinculado, se houver;
- tema visual;
- status;
- descrição curta.

Critérios de aceite

- O resumo deve aparecer mesmo em Projeto recém-criado.
- Campos opcionais ausentes não devem quebrar a interface.
- O usuário deve ter acesso fácil à edição/configuração do Projeto.

RF-025 — O Dashboard deve exibir indicadores básicos quando existirem dados

O Dashboard deve exibir indicadores simples, quando houver dados cadastrados.

Indicadores possíveis no MVP 1:

- quantidade de sessões;

- quantidade de personagens/criaturas;
- quantidade de locais;
- quantidade de facções;
- quantidade de documentos;
- quantidade de notas;
- quantidade de relações.

Critérios de aceite

- Indicadores devem exibir zero quando não houver dados.
- Indicadores não devem exigir módulos futuros.
- Os números devem refletir dados do Projeto atual.

RF-026 — O Dashboard deve destacar próxima ou última sessão

O Dashboard deve destacar, quando houver:

- próxima sessão planejada;
- última sessão jogada;
- data;
- status;
- resumo curto;
- botão para abrir a sessão.

Se não houver sessões cadastradas, o Dashboard deve exibir ação para criar a primeira sessão.

Critérios de aceite

- Projeto sem sessões deve mostrar estado vazio útil.
- Projeto com sessão planejada deve destacar a próxima sessão.
- Projeto com sessões jogadas deve permitir acesso à última sessão.
- O destaque deve respeitar o contexto do Projeto atual.

RF-027 — O Dashboard deve exibir atalhos para módulos ativos

O Dashboard deve mostrar atalhos para os módulos ativos do Projeto.

Módulos ocultos não devem aparecer como atalhos principais.

Critérios de aceite

- Atalhos devem abrir o módulo correspondente.
- Módulos ocultos não devem aparecer no destaque.
- O usuário deve ter acesso às configurações para reativar módulos ocultos.

RF-028 — O Dashboard deve exibir notas recentes do Projeto

O Dashboard deve exibir notas recentes associadas ao Projeto de Jogo.

O usuário deve poder acessar ou criar uma nota rapidamente.

Critérios de aceite

- Projeto sem notas deve mostrar estado vazio útil.
- Projeto com notas deve listar notas recentes.
- O usuário deve conseguir criar nota a partir do Dashboard.
- Notas exibidas devem pertencer ao Projeto atual.

RF-029 — O Dashboard deve respeitar o tema visual escolhido

O Dashboard deve aplicar o tema visual configurado no Projeto.

No MVP 1, o tema Cartógrafo deve estar funcional.

A estrutura deve permitir evolução para Sombrio/Terror e Futurista.

Critérios de aceite

- Projeto com tema Cartógrafo deve exibir identidade visual correspondente.
- O tema salvo no Projeto deve ser carregado no Dashboard.
- A aplicação de tema não deve alterar a estrutura de navegação.
- O tema não deve prejudicar legibilidade.

RF-030 — O Dashboard deve ter navegação lateral interna

Dentro de um Projeto de Jogo, o sistema deve exibir navegação lateral com os módulos internos ativos.

A navegação interna deve ser diferente da navegação global.

Módulos internos obrigatórios no MVP 1:

- Resumo;
- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Facções;
- Documentos;
- Notas;
- Relações;
- Configurações da Campanha.

Critérios de aceite

- A navegação interna deve aparecer dentro do Projeto.
- A navegação interna deve exibir apenas módulos ativos.
- O usuário deve conseguir alternar entre módulos.
- O usuário deve conseguir voltar para a Home/global.

RF-031 — O Dashboard deve ter busca interna simples do Projeto

Dentro de um Projeto de Jogo, o sistema deve oferecer busca interna simples.

No MVP 1, a busca interna pode procurar por:

- sessões;
- cenas;
- personagens/criaturas;
- locais;
- facções;
- documentos;
- notas.

Critérios de aceite

- A busca deve retornar apenas dados do Projeto atual.
- Quando não houver resultado, deve exibir estado vazio.
- A busca interna não deve misturar dados de outros Projetos.

6. Estados vazios, validações e feedbacks

RF-032 — O sistema deve exibir estados vazios úteis

Telas sem dados devem exibir estados vazios com orientação clara.

Exemplos:

- sem campanhas;
- sem sessões;
- sem personagens/criaturas;

- sem locais;
- sem facções;
- sem documentos;
- sem notas;
- busca sem resultados.

Critérios de aceite

- Estado vazio deve explicar o que está faltando.
- Estado vazio deve oferecer ação principal quando aplicável.
- Estado vazio não deve parecer erro técnico.

RF-033 — O sistema deve validar formulários principais

O sistema deve validar campos obrigatórios e formatos antes de salvar.

Formulários principais no MVP 1:

- cadastro;
- login;
- criação de Projeto de Jogo;
- edição de Projeto de Jogo;
- criação de nota;
- criação de registros nos módulos internos.

Critérios de aceite

- Campos obrigatórios devem indicar erro quando vazios.
- E-mail deve ter formato válido.
- O sistema deve impedir envio inválido quando possível.
- Mensagens de erro devem ser claras.

RF-034 — O sistema deve exibir feedback ao salvar dados

Ao criar, editar, arquivar ou excluir dados, o sistema deve informar o resultado da operação.

Feedbacks esperados:

- salvando;
- salvo com sucesso;
- erro ao salvar;
- removido;
- arquivado;
- erro de validação.

Critérios de aceite

- O usuário deve saber se a ação foi concluída.
- Erros devem ser exibidos de forma compreensível.
- O sistema não deve falhar silenciosamente.

RF-035 — O sistema deve preservar data de criação e atualização

Registros principais devem armazenar data de criação e data de última atualização.

Registros iniciais:

- usuário;
- Projeto de Jogo;
- mundo/cenário;
- documento;
- nota;
- sessão;
- cena;

- personagem/criatura;
- local;
- facção;
- relação.

Critérios de aceite

- Todo registro principal deve possuir data de criação.
- Registros editados devem atualizar data de modificação.
- A Home e listagens podem usar essas datas para ordenação.

RF-036 — O sistema deve preparar estrutura para papéis e visibilidade futura

Mesmo sem jogadores no MVP 1, o sistema deve prever campos ou estrutura para evolução futura.

Preparações necessárias:

- dono do Projeto;
- papel do usuário;
- visibilidade do conteúdo;
- status de cânone;
- status de revisão;
- origem do conteúdo.

Critérios de aceite

- O MVP não precisa oferecer interface completa de jogadores.
- A modelagem não deve impedir adição futura de jogadores.
- Conteúdos principais devem poder receber campos de visibilidade e cânone.

Requisitos Funcionais do MVP 1

Parte 2 — Módulos Internos Essenciais

1. Escopo desta seção

Esta seção define os requisitos funcionais dos módulos internos essenciais de um Projeto de Jogo no MVP 1.

Módulos cobertos nesta parte:

- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Facções;
- Documentos;
- Notas;
- Relações.

Módulos como Itens & Equipamentos, Magias & Poderes, Regras / Compêndio, Handouts, Pistas & Segredos, Recursos / Soundtracks e IA da Campanha ficam previstos para fases futuras ou versões simples posteriores.

2. Sessões

RF-037 — O usuário deve poder listar sessões de um Projeto de Jogo

O sistema deve permitir listar as sessões vinculadas ao Projeto de Jogo atual.

A listagem deve exibir:

- título;
- número ou ordem da sessão;
- data real, se informada;
- status;
- resumo curto;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas sessões do Projeto atual.
- Projeto sem sessões deve exibir estado vazio útil.
- O usuário deve conseguir abrir uma sessão da lista.
- A listagem deve permitir ordenação por data ou número da sessão.

RF-038 — O usuário deve poder criar uma sessão

O sistema deve permitir criar uma sessão dentro de um Projeto de Jogo.

Campos mínimos:

- título;
- status.

Campos opcionais:

- número ou ordem;
- data real;
- data fictícia;
- resumo;
- anotações brutas;
- objetivos;
- pendências;
- ganchos para próxima sessão.

Status iniciais:

- planejada;
- em preparação;
- agendada;
- jogada;
- resumida;
- revisada;
- cancelada;
- arquivada.

Critérios de aceite

- A sessão criada deve pertencer ao Projeto atual.
- O sistema deve permitir criar sessão com título e status.
- Se o usuário não definir status, o sistema deve usar “planejada” ou equivalente.
- A sessão criada deve aparecer na listagem de sessões.

RF-039 — O usuário deve poder editar uma sessão

O sistema deve permitir editar os dados de uma sessão.

Campos editáveis:

- título;
- número ou ordem;
- data real;
- data fictícia;
- status;
- resumo;
- anotações brutas;
- objetivos;
- pendências;
- ganchos;
- notas do mestre.

Critérios de aceite

- Alterações devem ser persistidas.
- A data de atualização deve ser alterada após edição.
- O sistema deve exibir feedback de sucesso ou erro.
- O usuário não deve editar sessões de outro Projeto.

RF-040 — O usuário deve poder arquivar ou excluir uma sessão

O sistema deve permitir arquivar uma sessão.

Exclusão permanente pode ser omitida no MVP ou tratada com confirmação explícita.

Critérios de aceite

- Sessão arquivada não deve aparecer na listagem principal por padrão.
- Sessões arquivadas devem poder ser acessadas por filtro, se implementado.
- A ação deve exibir confirmação ou feedback.
- O sistema não deve remover dados acidentalmente sem aviso.

3. Cenas

RF-041 — O usuário deve poder listar cenas de um Projeto de Jogo

O sistema deve permitir listar cenas vinculadas ao Projeto atual.

A listagem deve exibir:

- título;
- status;
- sessão vinculada, se houver;
- local vinculado, se houver;
- resumo curto;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas cenas do Projeto atual.
- Projeto sem cenas deve exibir estado vazio útil.
- O usuário deve conseguir abrir uma cena da lista.
- A listagem deve permitir filtro por status, se viável no MVP.

RF-042 — O usuário deve poder criar uma cena

O sistema deve permitir criar uma cena dentro de um Projeto de Jogo.

Campos mínimos:

- título;
- status.

Campos opcionais:

- resumo;
- objetivo da cena;
- descrição para o mestre;
- texto para ler aos jogadores;
- sessão vinculada;
- local vinculado;
- notas privadas;
- consequências possíveis.

Status iniciais:

- ideia;
- preparada;
- pronta para jogar;
- jogada;
- improvisada;
- pulada;
- descartada;
- reaproveitável;
- arquivada.

Critérios de aceite

- A cena criada deve pertencer ao Projeto atual.
- A cena deve poder existir sem sessão vinculada.
- A cena deve poder ser vinculada a uma sessão posteriormente.
- A cena criada deve aparecer na listagem de cenas.

RF-043 — O usuário deve poder editar uma cena

O sistema deve permitir editar os dados de uma cena.

Campos editáveis:

- título;
- status;
- resumo;
- objetivo;
- descrição;

- texto para jogadores;
- sessão vinculada;
- local vinculado;
- consequências possíveis;
- notas privadas.

Critérios de aceite

- Alterações devem ser persistidas.
- O sistema deve exibir feedback de salvamento.
- A data de atualização deve ser alterada.
- O usuário não deve editar cenas de outro Projeto.

RF-044 — O usuário deve poder arquivar uma cena

O sistema deve permitir arquivar cenas.

Critérios de aceite

- Cena arquivada não deve aparecer na listagem principal por padrão.
- A ação deve ter feedback.
- O usuário deve poder consultar cenas arquivadas por filtro ou seção, se implementado.

4. Personagens & Criaturas

RF-045 — O usuário deve poder listar Personagens & Criaturas

O sistema deve permitir listar registros do módulo Personagens & Criaturas vinculados ao Projeto atual.

A listagem deve exibir:

- nome;
- tipo principal;
- natureza/classificação, se informada;
- espécie/raça/povo/linhagem, se informada;
- status de cânone;
- visibilidade;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas registros do Projeto atual.
- Projeto sem registros deve exibir estado vazio útil.
- O usuário deve conseguir abrir um registro da lista.
- A listagem deve permitir filtro por tipo principal no MVP, se viável.

RF-046 — O usuário deve poder criar Personagem ou Criatura

O sistema deve permitir criar um registro no módulo Personagens & Criaturas.

Campos mínimos:

- nome;
- tipo principal.

Campos opcionais:

- natureza/classificação;
- espécie/raça/povo/linhagem;
- descrição pública;
- notas do mestre;
- status de cânone;
- visibilidade;
- status narrativo;

- tags.

Tipos principais iniciais:

- PJ;
- NPC;
- Criatura / Monstro;
- Ameaça natural;
- Ameaça mística / sobrenatural;
- Ameaça ambiental;
- Entidade única;
- Outro / Personalizado.

Critérios de aceite

- O registro deve pertencer ao Projeto atual.
- O sistema deve permitir criar registro com nome e tipo principal.
- O tipo principal deve ser editável.
- O registro criado deve aparecer na listagem do módulo.

RF-047 — O usuário deve poder editar Personagem ou Criatura

O sistema deve permitir editar dados de Personagens & Criaturas.

Campos editáveis:

- nome;
- tipo principal;
- natureza/classificação;
- espécie/raça/povo/linhagem;
- descrição pública;
- notas do mestre;
- status de cânone;
- visibilidade;
- status narrativo;
- tags.

Critérios de aceite

- Alterações devem ser persistidas.
- A data de atualização deve ser alterada.
- O sistema deve exibir feedback de sucesso ou erro.
- O usuário não deve editar registros de outro Projeto.

RF-048 — O usuário deve poder arquivar Personagem ou Criatura

O sistema deve permitir arquivar registros de Personagens & Criaturas.

Critérios de aceite

- Registro arquivado não deve aparecer na listagem principal por padrão.
- O usuário deve conseguir acessar registros arquivados por filtro ou seção, se implementado.
- A ação deve exibir feedback.

5. Locais / Atlas

RF-049 — O usuário deve poder listar locais do Projeto

O sistema deve permitir listar locais vinculados ao Projeto atual.

A listagem deve exibir:

- nome;
- tipo de local, se informado;

- local pai, se houver;
- status de cânone;
- visibilidade;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas locais do Projeto atual.
- Projeto sem locais deve exibir estado vazio útil.
- O usuário deve conseguir abrir um local da lista.
- A listagem deve permitir identificar hierarquia básica, quando houver local pai.

RF-050 — O usuário deve poder criar um local

O sistema deve permitir criar locais no módulo Locais / Atlas.

Campos mínimos:

- nome.

Campos opcionais:

- tipo de local;
- descrição;
- notas do mestre;
- local pai;
- status de cânone;
- visibilidade;
- imagem ou mapa simples;
- tags.

Tipos de local sugeridos:

- mundo;
- continente;
- região;
- reino;
- cidade;
- bairro;
- construção;
- sala;
- dungeon;
- plano;
- dimensão;
- nave;
- estação espacial;
- local personalizado.

Critérios de aceite

- O local criado deve pertencer ao Projeto atual.
- O local pode existir sem local pai.
- O local pode ser vinculado a outro local como local pai.
- O local criado deve aparecer na listagem.

RF-051 — O usuário deve poder editar um local

O sistema deve permitir editar dados de um local.

Campos editáveis:

- nome;
- tipo de local;
- descrição;

- notas do mestre;
- local pai;
- status de cânone;
- visibilidade;
- imagem ou mapa simples;
- tags.

Critérios de aceite

- Alterações devem ser persistidas.
- O sistema deve impedir hierarquia circular, se houver local pai.
- A data de atualização deve ser alterada.
- O sistema deve exibir feedback ao salvar.

RF-052 — O usuário deve poder arquivar um local

O sistema deve permitir arquivar locais.

Critérios de aceite

- Local arquivado não deve aparecer na listagem principal por padrão.
- O usuário deve conseguir acessar locais arquivados por filtro ou seção, se implementado.
- A ação deve exibir feedback.

6. Facções

RF-053 — O usuário deve poder listar facções

O sistema deve permitir listar facções vinculadas ao Projeto atual.

A listagem deve exibir:

- nome;
- tipo de facção, se informado;
- status de cânone;
- visibilidade;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas facções do Projeto atual.
- Projeto sem facções deve exibir estado vazio útil.
- O usuário deve conseguir abrir uma facção da lista.

RF-054 — O usuário deve poder criar uma facção

O sistema deve permitir criar uma facção.

Campos mínimos:

- nome.

Campos opcionais:

- tipo de facção;
- descrição pública;
- notas do mestre;
- objetivos;
- status de cânone;
- visibilidade;
- tags.

Tipos sugeridos:

- organização;

- clã;
- seita;
- culto;
- ordem;
- guilda;
- casa nobre;
- governo;
- corporação;
- facção criminosa;
- facção personalizada.

Critérios de aceite

- A facção criada deve pertencer ao Projeto atual.
- A facção criada deve aparecer na listagem de facções.
- A facção deve poder existir sem personagens vinculados.

RF-055 — O usuário deve poder editar uma facção

O sistema deve permitir editar dados da facção.

Campos editáveis:

- nome;
- tipo de facção;
- descrição pública;
- notas do mestre;
- objetivos;
- status de cânone;
- visibilidade;
- tags.

Critérios de aceite

- Alterações devem ser persistidas.
- A data de atualização deve ser alterada.
- O sistema deve exibir feedback de salvamento.

RF-056 — O usuário deve poder arquivar uma facção

O sistema deve permitir arquivar facções.

Critérios de aceite

- Facção arquivada não deve aparecer na listagem principal por padrão.
- A ação deve exibir feedback.
- Relações existentes com a facção não devem ser apagadas automaticamente.

7. Documentos

RF-057 — O usuário deve poder listar documentos do Projeto

O sistema deve permitir listar documentos vinculados ao Projeto atual.

A listagem deve exibir:

- título;
- tipo de documento;
- status;
- data de criação;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas documentos do Projeto atual.
- Projeto sem documentos deve exibir estado vazio útil.
- O usuário deve conseguir abrir ou visualizar detalhes do documento.

RF-058 — O usuário deve poder cadastrar um documento simples

No MVP 1, o sistema deve permitir cadastrar um documento como registro simples.

Campos mínimos:

- título;
- tipo.

Campos opcionais:

- descrição;
- link externo;
- arquivo anexado, se upload simples for implementado;
- notas;
- status;
- tags.

Tipos sugeridos:

- PDF;
- texto;
- mapa;
- imagem;
- handout;
- manual;
- cenário;
- aventura;
- crônica;
- resumo de sessão;
- outro.

Critérios de aceite

- Documento criado deve pertencer ao Projeto atual.
- O documento deve aparecer na listagem.
- O sistema não precisa extrair texto automaticamente no MVP 1.
- Upload de arquivo pode ser versão simples ou adiado, desde que o registro de documento exista.

RF-059 — O usuário deve poder editar documento

O sistema deve permitir editar dados de um documento.

Campos editáveis:

- título;
- tipo;
- descrição;
- link externo;
- arquivo anexado, se houver;
- notas;
- status;
- tags.

Critérios de aceite

- Alterações devem ser persistidas.
- A data de atualização deve ser alterada.
- O sistema deve exibir feedback ao salvar.

RF-060 — O usuário deve poder arquivar documento

O sistema deve permitir arquivar documentos.

Critérios de aceite

- Documento arquivado não deve aparecer na listagem principal por padrão.
- Documento arquivado não deve ser removido permanentemente.
- A ação deve exibir feedback.

8. Notas

RF-061 — O usuário deve poder listar notas do Projeto

O sistema deve permitir listar notas vinculadas ao Projeto atual.

A listagem deve exibir:

- título, se houver;
- trecho inicial;
- contexto, se houver;
- data de criação;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas notas do Projeto atual.
- Projeto sem notas deve exibir estado vazio útil.
- O usuário deve conseguir abrir uma nota da lista.

RF-062 — O usuário deve poder criar nota de Projeto

O sistema deve permitir criar nota vinculada ao Projeto atual.

Campos mínimos:

- conteúdo.

Campos opcionais:

- título;
- contexto;
- tags;
- status: ativa ou arquivada.

Critérios de aceite

- A nota criada deve pertencer ao Projeto atual.
- O usuário deve conseguir criar nota apenas com conteúdo.
- A nota criada deve aparecer na listagem de notas.

RF-063 — O usuário deve poder editar nota

O sistema deve permitir editar notas.

Campos editáveis:

- título;
- conteúdo;
- contexto;
- tags;
- status.

Critérios de aceite

- Alterações devem ser persistidas.

- A data de atualização deve ser alterada.
- O sistema deve exibir feedback ao salvar.

RF-064 — O usuário deve poder arquivar nota

O sistema deve permitir arquivar notas.

Critérios de aceite

- Nota arquivada não deve aparecer na listagem principal por padrão.
- A ação deve exibir feedback.
- A nota não deve ser removida permanentemente.

9. Relações básicas

RF-065 — O usuário deve poder listar relações do Projeto

O sistema deve permitir listar relações criadas dentro do Projeto atual.

A listagem deve exibir:

- entidade de origem;
- tipo de relação;
- entidade de destino;
- descrição curta;
- data da última atualização.

Critérios de aceite

- A lista deve exibir apenas relações do Projeto atual.
- Projeto sem relações deve exibir estado vazio útil.
- O usuário deve conseguir abrir ou editar uma relação.

RF-066 — O usuário deve poder criar relação entre conteúdos

O sistema deve permitir criar uma relação entre dois conteúdos do mesmo Projeto.

Campos mínimos:

- origem;
- tipo de relação;
- destino.

Campos opcionais:

- descrição;
- intensidade;
- status;
- visibilidade;
- notas do mestre.

Tipos de relação sugeridos:

- pertence a;
- controla;
- rivaliza com;
- serve a;
- protege;
- conhece;
- investiga;
- ameaça;
- vive em;
- ocorreu em;
- possui;

- criou;
- revelou;
- esconde;
- depende de;
- foi extraído de;
- outro / personalizado.

Critérios de aceite

- Origem e destino devem existir dentro do Projeto atual.
- A relação deve aparecer na listagem.
- O sistema deve permitir tipo personalizado ou lista básica editável.
- O sistema não deve permitir relação sem origem ou destino.

RF-067 — O usuário deve poder editar relação

O sistema deve permitir editar uma relação.

Campos editáveis:

- tipo de relação;
- descrição;
- intensidade;
- status;
- visibilidade;
- notas do mestre.

Critérios de aceite

- Alterações devem ser persistidas.
- A data de atualização deve ser alterada.
- O sistema deve exibir feedback ao salvar.

RF-068 — O usuário deve poder remover ou arquivar relação

O sistema deve permitir remover ou arquivar relações.

No MVP 1, a remoção pode ser permitida com confirmação, ou a relação pode ser arquivada.

Critérios de aceite

- O usuário deve confirmar a remoção, se ela for permanente.
- Relações arquivadas não devem aparecer na listagem principal por padrão.
- A ação deve exibir feedback.

10. Busca interna e filtros mínimos dos módulos

RF-069 — Módulos internos devem ter busca simples

Os módulos internos essenciais devem permitir busca simples por texto.

Módulos prioritários:

- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Facções;
- Documentos;
- Notas;
- Relações.

Critérios de aceite

- A busca deve funcionar dentro do Projeto atual.
- A busca deve retornar resultados do módulo atual.
- Busca sem resultados deve exibir estado vazio.

RF-070 — Módulos internos devem ter filtros mínimos quando aplicável

Alguns módulos devem permitir filtros básicos.

Filtros sugeridos no MVP 1:

Personagens & Criaturas:

- tipo principal;
- status de cânone;
- visibilidade.

Sessões:

- status.

Cenas:

- status.

Documentos:

- tipo;
- status.

Locais:

- tipo de local.

Facções:

- tipo de facção.

Critérios de aceite

- Filtros devem ser opcionais.
- Filtros não devem quebrar a listagem.
- O usuário deve conseguir limpar filtros aplicados.

11. Critérios de aceite desta parte

Para considerar esta parte implementada no MVP 1:

- usuário consegue criar, listar, editar e arquivar sessões;
- usuário consegue criar, listar, editar e arquivar cenas;
- usuário consegue criar, listar, editar e arquivar Personagens & Criaturas;
- usuário consegue criar, listar, editar e arquivar locais;
- usuário consegue criar, listar, editar e arquivar facções;
- usuário consegue criar, listar, editar e arquivar documentos simples;
- usuário consegue criar, listar, editar e arquivar notas;
- usuário consegue criar, listar, editar e remover/arquivar relações básicas;
- módulos exibem estados vazios úteis;
- módulos possuem busca simples;
- filtros mínimos existem onde forem prioritários;
- todos os registros pertencem ao Projeto atual;
- dados de um Projeto não aparecem indevidamente em outro.

Requisitos Não Funcionais do MVP 1

1. Objetivo

Este documento define os requisitos não funcionais do MVP 1 do Nat 1.

Enquanto os requisitos funcionais descrevem o que o sistema faz, os requisitos não funcionais descrevem como o sistema deve se comportar em termos de segurança, desempenho, organização técnica, manutenibilidade, responsividade e preparação para evolução futura.

O MVP 1 será uma aplicação web organizacional para mestres/narradores de RPG, com frontend em React + TypeScript + Vite e backend em FastAPI + PostgreSQL.

2. Segurança e privacidade

RNF-001 — O sistema deve proteger rotas privadas

Todas as telas internas do Nat 1 devem exigir autenticação.

Rotas como Home do Mestre, Campanhas & Crônicas, Dashboard da Campanha e módulos internos não podem ser acessadas por usuários não autenticados.

RNF-002 — O sistema deve isolar dados por usuário

No MVP 1, cada usuário deve visualizar apenas seus próprios dados.

Um usuário não deve conseguir acessar campanhas, mundos, documentos, notas, personagens, sessões ou qualquer outro conteúdo pertencente a outro usuário.

RNF-003 — Senhas não devem ser armazenadas em texto puro

Senhas devem ser armazenadas usando hash seguro.

O sistema nunca deve salvar senha original no banco de dados.

RNF-004 — Dados sensíveis não devem ser expostos no frontend

Tokens, chaves, segredos, variáveis de ambiente e credenciais não devem ser expostos no frontend nem commitados no repositório.

Arquivos .env devem ficar fora do Git.

RNF-005 — O sistema deve validar dados recebidos no backend

O backend deve validar dados recebidos pela API, mesmo que o frontend também valide formulários.

A validação deve usar schemas adequados, preferencialmente com Pydantic no FastAPI.

RNF-006 — O sistema deve tratar erros sem expor detalhes internos

Erros de API não devem revelar informações sensíveis, como stack trace, credenciais, estrutura interna do banco ou detalhes de infraestrutura.

Mensagens para o usuário devem ser compreensíveis e seguras.

3. Performance e experiência de uso

RNF-007 — O sistema deve ter boa resposta nas operações principais

Operações comuns do MVP devem responder rapidamente em ambiente normal de uso.

Operações principais:

- login;
- carregamento da Home do Mestre;

- listagem de campanhas;
- abertura do Dashboard;
- listagem de módulos;
- criação e edição de registros simples.

Como meta inicial, essas operações devem responder em tempo adequado para uma aplicação web comum, sem travamentos perceptíveis.

RNF-008 — O frontend deve evitar carregamentos desnecessários

O frontend deve buscar apenas os dados necessários para a tela atual.

A aplicação não deve carregar todos os dados de todas as campanhas ao abrir a Home.

Dados internos de uma campanha devem ser carregados apenas quando o usuário entrar nela ou acessar seus módulos.

RNF-009 — O sistema deve exibir estados de carregamento

Telas e ações assíncronas devem exibir feedback visual.

Exemplos:

- carregando;
- salvando;
- excluindo;
- arquivando;
- nenhum resultado encontrado;
- erro ao carregar.

O usuário não deve ficar sem resposta visual durante operações importantes.

RNF-010 — O sistema deve funcionar prioritariamente bem em desktop e notebook

O Nat 1 será usado principalmente por mestres durante preparação e narração, geralmente em desktop ou notebook.

O MVP 1 deve priorizar boa experiência em telas médias e grandes.

RNF-011 — O sistema deve ser responsivo para telas menores

Mesmo com foco em desktop, o sistema deve ser minimamente responsivo.

Ele deve funcionar de forma aceitável em tablets e telas menores, sem quebrar layout, gerar overflow horizontal ou impedir uso básico.

4. Acessibilidade e usabilidade

RNF-012 — A interface deve ter contraste adequado

Textos, botões, cards e links devem manter contraste suficiente para leitura confortável.

Temas visuais não devem prejudicar legibilidade.

RNF-013 — O sistema deve usar elementos semânticos sempre que possível

A interface deve usar estrutura HTML adequada.

Exemplos:

- botões para ações;
- links para navegação;
- headings organizados;
- labels em campos de formulário;
- landmarks quando aplicável.

RNF-014 — Formulários devem ser claros e previsíveis

Campos obrigatórios devem ser identificáveis.

Erros de validação devem indicar o que precisa ser corrigido.

O usuário deve entender por que uma ação falhou.

RNF-015 — A navegação deve manter contexto claro

O usuário deve saber se está no nível global do sistema ou dentro de uma campanha/crônica.

A navegação global e a navegação interna da campanha devem ser visualmente distinguíveis.

5. Manutenibilidade e organização do código

RNF-016 — O projeto deve usar TypeScript no frontend

O frontend deve ser implementado com TypeScript para melhorar segurança, previsibilidade e manutenção do código.

RNF-017 — O backend deve usar tipagem e schemas claros

O backend deve usar tipagem Python e schemas Pydantic para entrada e saída de dados da API.

Isso melhora documentação, validação e manutenção.

RNF-018 — O código deve ser organizado por responsabilidade

O projeto deve evitar arquivos grandes demais e mistura de responsabilidades.

No frontend, separar:

- páginas;
- componentes;
- hooks;
- serviços de API;
- tipos;
- estilos/tokens.

No backend, separar:

- rotas;
- models;
- schemas;
- services;
- repositórios ou camada de acesso a dados;
- configurações;
- autenticação.

RNF-019 — O banco deve usar migrations versionadas

Alterações no banco de dados devem ser feitas por migrations.

A tecnologia aprovada para isso é Alembic.

RNF-020 — A API deve ter documentação automática

A API do backend deve aproveitar a documentação automática do FastAPI/OpenAPI.

Isso ajudará no desenvolvimento, testes e integração com o frontend.

RNF-021 — O sistema deve seguir padrão consistente de nomes

O projeto deve manter consistência entre nomes técnicos e nomes da interface.

Exemplos:

- Interface: Campanhas & Crônicas;
- Conceito técnico: Projeto de Jogo;
- Código/API: `game_project` ou equivalente definido;
- Interface: Personagens & Criaturas;

- Código/API: entidade ou módulo equivalente.
- A documentação deve registrar essas equivalências.

6. Banco de dados e integridade

RNF-022 — Registros principais devem ter identificadores únicos

Entidades principais devem possuir identificador único.

Exemplos:

- usuário;
- projeto de jogo;
- mundo/cenário;
- sistema/template;
- sessão;
- cena;
- personagem/criatura;
- local;
- facção;
- documento;
- nota;
- relação.

RNF-023 — Registros principais devem guardar data de criação e atualização

Registros principais devem possuir:

- data de criação;
- data de última atualização.

Esses campos serão usados para ordenação, histórico básico e listagens recentes.

RNF-024 — Exclusões permanentes devem ser evitadas no MVP

No MVP 1, o sistema deve priorizar arquivamento em vez de exclusão definitiva.

Isso reduz risco de perda acidental de dados importantes da campanha.

Quando houver exclusão permanente, deve haver confirmação explícita.

RNF-025 — Relações devem preservar integridade mínima

Relações entre conteúdos devem apontar para registros existentes.

O sistema não deve permitir criar relação sem origem, destino ou tipo.

Quando um registro relacionado for arquivado, suas relações não devem ser apagadas automaticamente.

7. Escalabilidade futura

RNF-026 — A arquitetura deve permitir evolução para IA/RAG

Mesmo sem IA pesada no MVP 1, a arquitetura deve permitir evolução futura para:

- embeddings;
- pgvector;
- busca semântica;
- perguntas à campanha;
- extração de PDF;
- importação inteligente;
- classificação automática;
- revisão de sugestões.

O MVP não precisa implementar essas funcionalidades, mas não deve bloquear sua inclusão.

RNF-027 — A arquitetura deve permitir papéis e permissões futuras

Mesmo sem jogadores no MVP 1, a estrutura deve permitir evolução futura para:

- mestre;
- jogador;
- co-mestre;
- convidado;
- permissões por campanha;
- visibilidade por jogador/personagem.

O MVP pode trabalhar apenas com dono da campanha, mas não deve criar uma estrutura impossível de expandir.

RNF-028 — A arquitetura deve permitir temas visuais

O sistema deve ser preparado para temas visuais configuráveis.

Temas definidos na direção do produto:

- Cartógrafo;
- Sombrio/Terror;
- Futurista.

No MVP 1, o tema Cartógrafo será prioridade. A estrutura deve permitir adicionar ou completar os demais temas sem reescrever componentes.

RNF-029 — A arquitetura deve permitir módulos ocultáveis

A estrutura deve permitir ativar ou ocultar módulos por Projeto de Jogo.

O sistema não deve assumir que toda campanha sempre usará todos os módulos.

RNF-030 — A arquitetura deve permitir campos flexíveis no futuro

Como o Nat 1 precisa suportar vários sistemas de RPG e sistemas autorais, a arquitetura deve permitir expansão futura para campos personalizados.

No MVP 1, os campos podem ser mais simples, mas a modelagem não deve impedir templates editáveis e campos customizados no futuro.

8. Arquivos e documentos

RNF-031 — Upload de arquivos deve ser tratado com cautela

No MVP 1, o upload pode ser simples ou adiado.

Quando implementado, deve considerar:

- limite de tamanho;
- tipos permitidos;
- armazenamento seguro;
- associação ao usuário;
- associação ao Projeto de Jogo;
- validação básica do arquivo.

RNF-032 — O sistema deve diferenciar documento cadastrado de documento processado

No MVP 1, um documento pode existir apenas como registro simples.

Processamento automático, extração de texto, OCR, chunking e vetorização pertencem a fases futuras.

A estrutura deve permitir que um documento tenha status, como:

- cadastrado;
- ativo;
- arquivado;
- pendente de extração;

- extraído;
- processado;
- erro de processamento.

9. Logs, erros e observabilidade básica

RNF-033 — O backend deve registrar erros relevantes

O backend deve registrar erros relevantes para depuração.

No MVP, logs simples já são suficientes.

Os logs não devem incluir senhas, tokens ou dados sensíveis desnecessários.

RNF-034 — O sistema deve ter tratamento básico de erros de API

O frontend deve tratar respostas de erro da API.

Exemplos:

- erro de autenticação;
- erro de validação;
- registro não encontrado;
- erro de permissão;
- erro interno.

O usuário deve receber uma mensagem compreensível.

10. Deploy e ambiente

RNF-035 — O projeto deve separar configurações por ambiente

O sistema deve permitir configuração por ambiente.

Ambientes esperados:

- desenvolvimento local;
- homologação/teste;
- produção futura.

Variáveis de ambiente devem ser usadas para configurações sensíveis.

RNF-036 — O projeto deve ser executável localmente

O projeto deve ter instruções claras para rodar localmente.

A documentação deve incluir:

- requisitos;
- instalação;
- criação de ambiente;
- execução do frontend;
- execução do backend;
- configuração do banco;
- execução das migrations.

RNF-037 — O repositório deve conter .gitignore adequado

O projeto deve ignorar arquivos sensíveis ou gerados automaticamente.

Exemplos:

- .env;
- ambientes virtuais;
- node_modules;
- arquivos temporários;
- caches;

- builds;
- uploads locais, se aplicável.

11. Testabilidade mínima

RNF-038 — O projeto deve permitir testes progressivos

O MVP deve ser desenvolvido de forma que permita testes por partes.

Prioridades iniciais:

- validações de backend;
- autenticação;
- criação/listagem de Projetos de Jogo;
- módulos principais;
- permissões básicas por usuário.

Não é obrigatório ter cobertura alta no início, mas a estrutura deve permitir adicionar testes.

RNF-039 — Regras críticas devem ser testáveis

Regras críticas devem ser implementadas de forma testável.

Exemplos:

- usuário não acessa dados de outro usuário;
- rota privada exige autenticação;
- Projeto de Jogo pertence ao usuário correto;
- relação exige origem e destino;
- campos obrigatórios são validados.

12. Critérios gerais de aceite dos RNF

Para o MVP 1 ser considerado tecnicamente aceitável:

- rotas internas exigem autenticação;
- dados são isolados por usuário;
- senhas não são salvas em texto puro;
- frontend usa TypeScript;
- backend usa FastAPI com schemas Pydantic;
- banco usa PostgreSQL e migrations;
- sistema possui estados de carregamento, erro e vazio;
- interface é legível e responsiva o suficiente para desktop/notebook;
- projeto roda localmente com instruções claras;
- .env e segredos não entram no repositório;
- arquitetura não bloqueia IA, RAG, jogadores, permissões, temas e módulos futuros.

Modelagem Inicial de Dados do MVP 1

Revisão baseada em modularização de ferramentas de campanha/worldbuilding

1. Objetivo da revisão

Esta revisão ajusta a modelagem inicial do Nat 1 para ficar mais coerente com ferramentas consolidadas de campanha e worldbuilding, sem quebrar as decisões já aprovadas.

A estrutura continua baseada em:

- Home do Mestre;
- Campanhas & Crônicas como nome amigável;
- Projeto de Jogo como conceito técnico;
- Mundo/Cenário como camada de worldbuilding;
- módulos internos ativáveis/desativáveis;
- Personagens & Criaturas como módulo amplo;
- Locais / Atlas;
- Organizações / Facções;
- Documentos;
- Notas;
- Relações;
- preparação futura para IA, RAG, jogadores, permissões, mapas avançados, calendários e timelines.

2. Ajuste conceitual principal

2.1. De “módulos fixos” para “categorias modulares”

A modelagem deve tratar cada grande área da campanha como uma categoria/módulo configurável.

Cada Projeto de Jogo pode ativar ou ocultar módulos.

Exemplos de módulos principais:

- Resumo;
- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Organizações / Facções;
- Itens & Equipamentos;
- Magias & Poderes / Habilidades;
- Regras / Compêndio;
- Documentos;
- Notas;
- Relações;
- Mapas;
- Diários / Registros;
- Eventos;
- Calendários;
- Linhas do Tempo;
- Recursos / Soundtracks;
- IA da Campanha;
- Configurações.

No MVP 1, nem todos precisam ter CRUD completo.

2.2. Módulos obrigatórios do MVP 1

Módulos com implementação funcional no MVP 1:

- Resumo;

- Sessões;
- Cenas;
- Personagens & Criaturas;
- Locais / Atlas;
- Organizações / Facções;
- Documentos;
- Notas;
- Relações;
- Configurações da Campanha.

2.3. Módulos planejados para fases futuras

Módulos que devem ficar planejados na modelagem, mas não precisam ser completos no MVP 1:

- Itens & Equipamentos;
- Magias & Poderes / Habilidades;
- Regras / Compêndio;
- Mapas avançados;
- Handouts;
- Pistas & Segredos;
- Diários / Registros;
- Eventos;
- Calendários;
- Linhas do Tempo;
- Recursos / Soundtracks;
- IA da Campanha.

3. Entidades principais revisadas

3.1. users

Representa o usuário autenticado.

Campos

```
id
name
email
password_hash
created_at
updated_at
is_active
```

Observações

- email deve ser único.
- Senha deve ser salva apenas como hash.
- No MVP 1, cada usuário acessa apenas seus próprios dados.

3.2. game_projects

Representa uma Campanha/Crônica/Projeto de Jogo.

Campos

```
id
owner_user_id
name
slug
```

format
description
 status
 system_template_id
 world_id
theme
 cover_image_url
 metadata_json
 created_at
 updated_at
 archived_at

Ajustes

Adicionar metadata_json para guardar configurações específicas sem criar colunas demais cedo.

Exemplos de uso futuro:

```
{  
  "display_labels": {  
    "project": "Crônica",  
    "gm": "Narrador",  
    "party": "Coterie"  
  },  
  "preferred_dashboard_layout": "cartographer_default"  
}
```

Constraints recomendadas

unique(owner_user_id, slug)

3.3. worlds

Representa Mundo/Cenário.

Campos

 id
 owner_user_id
 name
slug
description
 status
 metadata_json
 created_at
 updated_at
 archived_at

Ajuste

Adicionar metadata_json para permitir worldbuilding avançado no futuro sem inflar a tabela principal.

Exemplos de conteúdo futuro:

```
{  
  "calendar_name": "Calendário de Astalon",
```

```
"cosmology_type": "steampunk dark fantasy",
"main_eras": ["Era Primordial", "Anos Sombrios", "Era da Tecnologia"]
}
```

Observações

Mundo/Cenário pode começar simples, mas deve estar preparado para:

- culturas;
- calendários;
- mitologia;
- línguas;
- povos;
- deuses;
- religiões;
- cosmologia;
- mapas;
- história;
- regiões;
- cidades;
- eventos históricos.

Constraints recomendadas

unique(owner_user_id, slug)

3.4. system_templates

Representa sistema/template de RPG.

Campos

```
id
owner_user_id
name
slug
type
description
is_builtin
metadata_json
created_at
updated_at
archived_at
```

Ajuste essencial

Adicionar metadata_json.

Exemplos:

```
{
"labels": {
"project": "Campanha",
"gm": "Mestre",
"character": "Personagem"
},
"suggested_modules": [
```

```

"sessions",
"scenes",
"characters_creatures",
"locations_atlas",
"organizations_factions"
],
"creature_natures": [
"Humanoide",
"Dragão",
"Morto-vivo",
"Besta"
]
}

```

Observações

Templates são editáveis e não devem engessar o mestre.

3.5. project_module_settings

Representa módulos ativos ou ocultos dentro de um Projeto de Jogo.

Campos

```

id
game_project_id
module_key
display_name
is_enabled
order_index
icon_key
created_at
updated_at

```

Ajustes

Adicionar:

```

display_name
icon_key

```

Isso permite que o mestre altere nome e ícone do módulo futuramente sem quebrar a estrutura.

```

module_key  recomendados

```

overview

sessions

scenes

```

characters_creatures
locations_atlas
organizations_factions

```

documents

notes

relationships

settings
 items_equipment
 powers_abilities
 rules_compendium
maps
handouts
 clues_secrets
 journals_logs
events
calendars
timelines
 resources_soundtracks
 campaign_ai
custom
Constraint recomendada
unique(game_project_id, module_key)

4. Conteúdos internos do Projeto de Jogo

4.1. sessions

Representa sessão real de jogo.

Campos

id
game_project_id
title
 session_number
 real_date
 fictional_date
 status
summary
 raw_notes
objectives
 pending_notes
 next_hooks
 gm_notes
 created_at
 updated_at
 archived_at

Observações

- session_number é opcional.
- Pode existir sem data.
- Continua como módulo central do MVP 1.

4.2. scenes

Representa cenas preparadas, jogadas, puladas ou improvisadas.

Campos

id
game_project_id
session_id
location_id
title
status
summary
objective
gm_description
player_text
possible_consequences
private_notes
created_at
updated_at
archived_at

Observações

- session_id opcional.
- location_id opcional.
- Relações adicionais devem usar relationships.

4.3. character_creatures

Representa Personagens & Criaturas.

Campos

id
game_project_id
world_id
name
main_type
nature_classification
species_lineage
narrative_status
canon_status
visibility
public_description
gm_notes
source_type
source_document_id
metadata_json
created_at
updated_at
archived_at

Ajustes

Adicionar:

world_id

metadata_json

world_id permite que um personagem/criatura pertença ao mundo/cenário e seja usado em várias campanhas no futuro.

metadata_json permite dados específicos por sistema.

main_type

player_character

npc

creature_monster

natural_threat

mystic_supernatural_threat

environmental_threat

unique_entity

custom

nature_classification

Categoria fundamental conforme sistema/template.

Exemplos:

Humanoide

Dragão

Morto-vivo

Cainita

Lupino

Espírito

Entidade paranormal

Constructo

Alienígena

species_lineage

Espécie, raça, povo ou linhagem.

Exemplos:

Humano

Elfo

Anão

Skarn

Interrano

K'tharriano

Brujah

Nosferatu

Garou

Observações

- Tipo principal, natureza/classificação e espécie/linhagem devem permanecer separados.
- Classificações devem vir do sistema/template ou do mestre.
- metadata_json pode guardar classe, clã, geração, ocupação, nível, NEX, trilha, atributos etc.

4.4. locations

Representa Locais / Atlas.

Campos

id
owner_user_id
game_project_id
world_id
parent_location_id
name
location_type
description
gm_notes
canon_status
visibility
image_url
metadata_json
created_at
updated_at
archived_at

Ajustes

Adicionar:

owner_user_id
world_id
metadata_json

Permitir que um local pertença a:

- um Projeto de Jogo;
- um Mundo/Cenário;
- ou ambos.

Regra de contexto

Um local deve possuir pelo menos um dos contextos:

game_project_id
world_id

Observações

- parent_location_id permite Atlas hierárquico.
- O backend deve impedir hierarquia circular.
- metadata_json pode guardar população, clima, bioma, coordenadas ficcionais, perigo, governo, economia etc.

4.5. organizations_factions

Substitui o nome técnico factions.

Por que ajustar o nome

“Facção” é correto, mas limitado. Ferramentas de campanha/worldbuilding costumam usar “Organisations/Organizações” como categoria ampla.

O Nat 1 deve exibir o módulo como:

Organizações / Facções

ou, em campanha mais simples:

Facções

Campos

id
owner_user_id
game_project_id
world_id
name
organization_type
public_description
gm_notes
goals
 canon_status
visibility
 metadata_json
 created_at
 updated_at
 archived_at
 organization_type sugeridos
organization
faction
clan
sect
cult
order
guild
 noble_house
family
government
corporation
 criminal_faction
 alien_faction
religion
custom

Ajustes

Adicionar:

owner_user_id
world_id
metadata_json

Observações

- Organizações/Facções podem existir no Mundo ou no Projeto.
- Famílias podem ser tratadas como tipo de Organização no MVP, sem criar tabela própria.
- Religiões também podem começar como Organização, mas no módulo Mundo/Cenário avançado podem ganhar estrutura própria.

4.6. documents

Representa documentos cadastrados.

Campos

id
owner_user_id
game_project_id
world_id
system_template_id
title
document_type
description
external_url
file_url
status
source_context
metadata_json
created_at
updated_at
archived_at

Ajustes

Adicionar:

owner_user_id
world_id
system_template_id
metadata_json

Observações

Documento pode ser:

- global;
- de sistema;
- de mundo;
- de campanha;
- vinculado a sessão/cena/entidade via relacionamento.

No MVP, documento pode ser registro simples.

Extração, OCR, RAG e importação ficam para fases futuras.

4.7. notes

Representa notas globais, de mundo ou de Projeto.

Campos

id
owner_user_id
game_project_id
world_id
title
content
context
status
created_at

updated_at
archived_at

Ajuste

Adicionar:

world_id

Observações

- Se game_project_id e world_id forem nulos, a nota é global.
- Notas podem virar conteúdo estruturado no futuro.

4.8. relationships

Representa relações entre conteúdos.

Campos

id
owner_user_id
game_project_id
world_id
source_type
source_id
target_type
target_id
relationship_type

description

intensity

status

visibility

created_at

updated_at

archived_at

Ajustes

Adicionar:

owner_user_id

world_id

Tipos possíveis

belongs_to

controls

rivals

serves

protects

knows

investigates

threatens

lives_in

occurred_in

possesses

created
revealed
hides
 depends_on
 extracted_from
 parent_of
 member_of
 located_in
custom

Observações técnicas

O modelo polimórfico será mantido no MVP, usando:

 source_type
 source_id
 target_type
 target_id

Isso exige validação forte no backend.

Índices recomendados

relationship(game_project_id)

relationship(world_id)

relationship(source_type, source_id)

relationship(target_type, target_id)

5. Módulos planejados, mas não obrigatórios no MVP 1

5.1. journals_logs

Representa Diários / Registros.

Uso futuro

Pode servir para:

- diário de personagem;
- diário de campanha;
- recap de sessão;
- registro de bordo;
- jornal da última sessão;
- crônica editorial;
- resumo em formato narrativo.

Campos futuros sugeridos

 id
 owner_user_id
 game_project_id
 world_id
 session_id
title
 journal_type
content
 status
visibility
 created_at

updated_at
archived_at

Observação

Não é obrigatório no MVP 1. Pode começar como notas/sessões.

5.2. events

Representa Eventos.

Uso futuro

Pode servir para:

- acontecimento histórico;
- evento de sessão;
- evento de mundo;
- evento político;
- evento cósmico;
- evento de calendário;
- consequência de ação dos jogadores.

Campos futuros sugeridos

id
owner_user_id
game_project_id
world_id
session_id

title

event_type

description

event_date_real
event_date_fictional

scope

status

visibility

created_at
updated_at
archived_at

Observação

Não entra no MVP 1 como CRUD obrigatório.

Pode ser derivado de sessões/cenas/relações inicialmente.

5.3. calendars

Representa Calendários.

Uso futuro

Pode servir para:

- calendário fictício;
- meses;
- semanas;
- estações;
- fases lunares;
- eventos marcados;
- datas históricas.

Observação

Fase 2 ou posterior.

5.4. timelines

Representa Linhas do Tempo.

Uso futuro

Pode servir para:

- história do mundo;
- história de facção;
- arco de personagem;
- cronologia da campanha;
- eras;
- eventos importantes.

Observação

Fase 2 ou posterior.

5.5. maps

No MVP 1, mapa pode ser `document_type = map` ou `locations.image_url`.

Em fase posterior, criar tabela própria.

Campos futuros sugeridos

`id`
`owner_user_id`
`game_project_id`
`world_id`

`title`

`description`

`image_url`
`map_type`

`visibility`

`created_at`
`updated_at`
`archived_at`

Depois:

`map_pins`
`map_layers`

5.6. abilities_powers

Pode representar Habilidades / Poderes / Magias.

Uso futuro

- magias;
- poderes;
- disciplinas;
- rituais;
- habilidades;
- técnicas;
- dons;
- poderes psíquicos.

No MVP 1, fica planejado dentro de Regras/Compêndio futuro.

5.7. items_equipment

Pode representar Itens & Equipamentos.

Uso futuro

- armas;
- armaduras;
- relíquias;
- consumíveis;
- equipamentos;
- tecnologia;
- veículos;
- artefatos;
- itens mágicos;
- itens homebrew.

No MVP 1, pode ficar fora ou ser tratado como nota/documento.

5.8. quests_objectives

Pode representar Missões / Objetivos.

Por que planejar

Ferramentas de campanha costumam ter categoria própria para quests/missões.

No Nat 1, isso pode ser útil para fantasia, sci-fi e campanhas abertas.

Uso futuro

- missão principal;
- missão secundária;
- objetivo de personagem;
- tarefa de facção;
- investigação em aberto;
- recompensa;
- status de conclusão.

No MVP 1, pode ser coberto por Sessões, Cenas, Notas e Pendências.

6. Tags e coleções

6.1. tags

Campos

id
owner_user_id
game_project_id
world_id
name

color

created_at
updated_at

Observações

- Opcional no primeiro corte do MVP.
- Útil para filtros e organização.

6.2. entity_tags

Campos

id
tag_id
target_type
target_id
created_at

Observações

- Relação polimórfica simples.
- Pode ser implementada após CRUD principal.

6.3. collections

Representa pastas/coleções personalizadas.

Campos

id
owner_user_id
game_project_id
world_id
name
description
module_key
icon_key
created_at
updated_at
archived_at

Observações

- Ajuda a criar categorias próprias.
- Não substitui relações.
- Pode entrar no MVP 1.2 se o MVP principal ficar pesado.

6.4. collection_items

Campos

id
collection_id
target_type
target_id
order_index
created_at

7. Modelo revisado final do MVP 1

Obrigatórios no MVP 1

users
game_projects
worlds

- system_templates
- project_module_settings
- sessions
- scenes
 - character_creatures
- locations
 - organizations_factions
- documents
- notes
- relationships
- Opcionais no MVP 1.2
- tags
 - entity_tags
- collections
 - collection_items
- Planejados para fases futuras
 - journals_logs
- events
- calendars
- timelines
- maps
 - map_pins
 - map_layers
 - items_equipment
 - abilities_powers
 - quests_objectives
 - custom_field_definitions
 - custom_field_values
 - document_chunks
 - ai_interactions
 - import_suggestions
 - project_members
- permissions
- messages
 - audit_logs

8. Constraints e índices recomendados

Constraints

users.email unique

game_projects(owner_user_id, slug) unique

worlds(owner_user_id, slug) unique

project_module_settings(game_project_id, module_key) unique

Para system_templates:

unique(owner_user_id, slug)

com cuidado especial para templates built-in.

Índices principais

game_projects.owner_user_id
worlds.owner_user_id
system_templates.owner_user_id
sessions.game_project_id
scenes.game_project_id
character_creatures.game_project_id
character_creatures.world_id
locations.game_project_id
locations.world_id
organizations_factions.game_project_id
organizations_factions.world_id
documents.owner_user_id
documents.game_project_id
documents.world_id
documents.system_template_id
notes.owner_user_id
notes.game_project_id
notes.world_id
relationships.owner_user_id
relationships.game_project_id
relationships.world_id
relationships(source_type, source_id)
relationships(target_type, target_id)

9. Decisões mantidas

Esta revisão mantém as decisões já aprovadas:

- não usar uma tabela genérica entities no MVP;
- usar tabelas claras por módulo principal;
- usar relação polimórfica controlada;
- separar tipo principal, natureza/classificação e espécie/raça/povo/linhagem;
- permitir módulos ativáveis/desativáveis;
- manter Mundo/Cenário como camada separada;
- manter Sistema como regras/templates;
- manter Documentos como fonte bruta;
- preparar IA/RAG para fases futuras;
- preparar jogadores/permissões para fases futuras;
- manter coleções personalizadas como organização auxiliar, não substituta de relações.

10. Veredito técnico

A modelagem revisada está mais coerente com ferramentas consolidadas de worldbuilding e campanha, sem copiar sua arquitetura diretamente.

Ela mantém o Nat 1 simples o suficiente para MVP, mas preparado para crescer em direção a:

- mundo vivo;
- módulos customizados;
- mapas;

- calendários;
- timelines;
- eventos;
- diários/recaps;
- compêndios;
- importação inteligente;
- IA contextual;
- jogadores e permissões.

Checklist de Pronto para Codar

- Criar repositório e estrutura de monorepo ou pastas frontend/backend.
- Configurar backend FastAPI, ambiente Python, SQLAlchemy, Alembic e PostgreSQL.
- Configurar frontend React + TypeScript + Vite + Tailwind.
- Implementar autenticação básica e isolamento por usuário.
- Criar migrations iniciais das tabelas obrigatórias do MVP 1.
- Implementar CRUD de Projetos de Jogo e Home do Mestre.
- Implementar Dashboard e módulos internos essenciais por etapas.
- Implementar feedbacks, estados vazios, validações e proteção de rotas.
- Manter tags/coleções, IA/RAG, eventos, mapas avançados e jogadores para fases posteriores.

Nota final: Este documento é a base oficial v1.0 para iniciar o desenvolvimento. Ajustes de implementação devem ser versionados e documentados sem alterar decisões aprovadas sem nova validação.